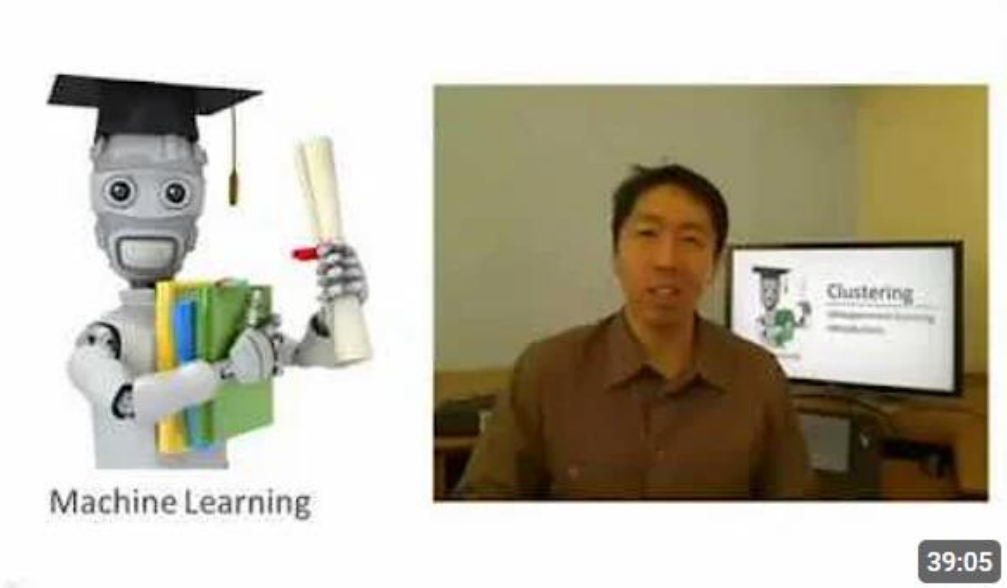
Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering → **today's lab**
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

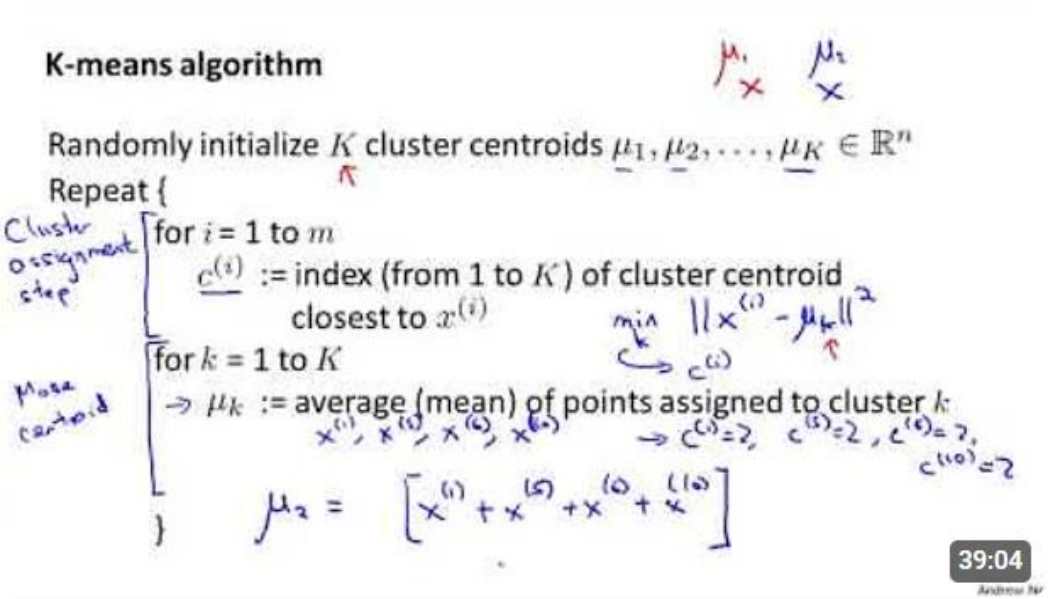


Lecture 08-01 Clustering

1,8 K de vizionări • acum 7 ani

Blitz Kim

Machine Learning by **Andrew Ng** [Coursera] 0801 Unsupervised learning introduction 0802 K-means algorithm 0803 Optimization



Clustering | ML-005 Lecture 13 | Stanford University | Andrew Ng

6,7 K de vizionări • acum 7 ani

Machine Learning and AI

Contents: Unsupervised Learning - Introduction, K-Means Algorithm, Optimization Objective, Random Initialization, Choosing the ...

Content

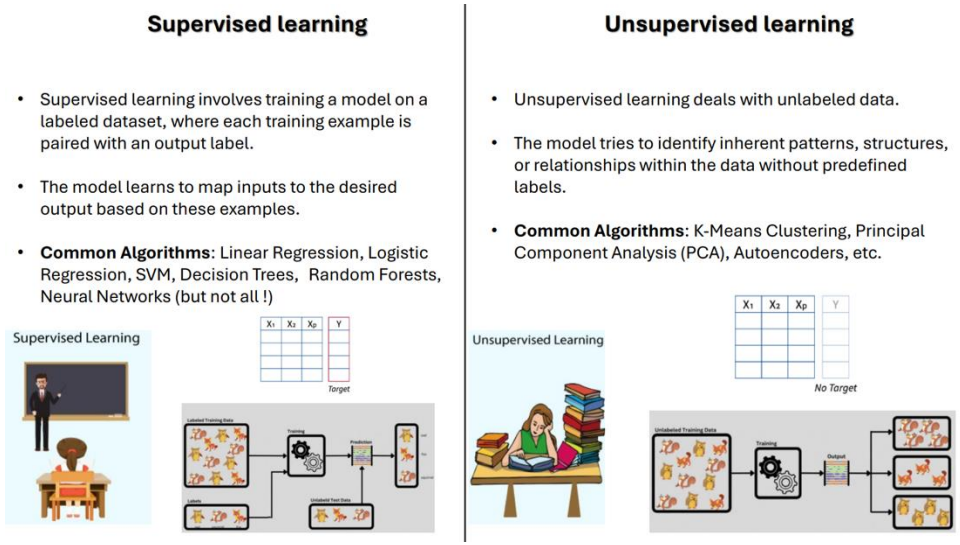
- Clustering - overview
- *K*-means clustering
- Agglomerative clustering
- DBSCAN
- Evaluating clustering methods

1) Clustering

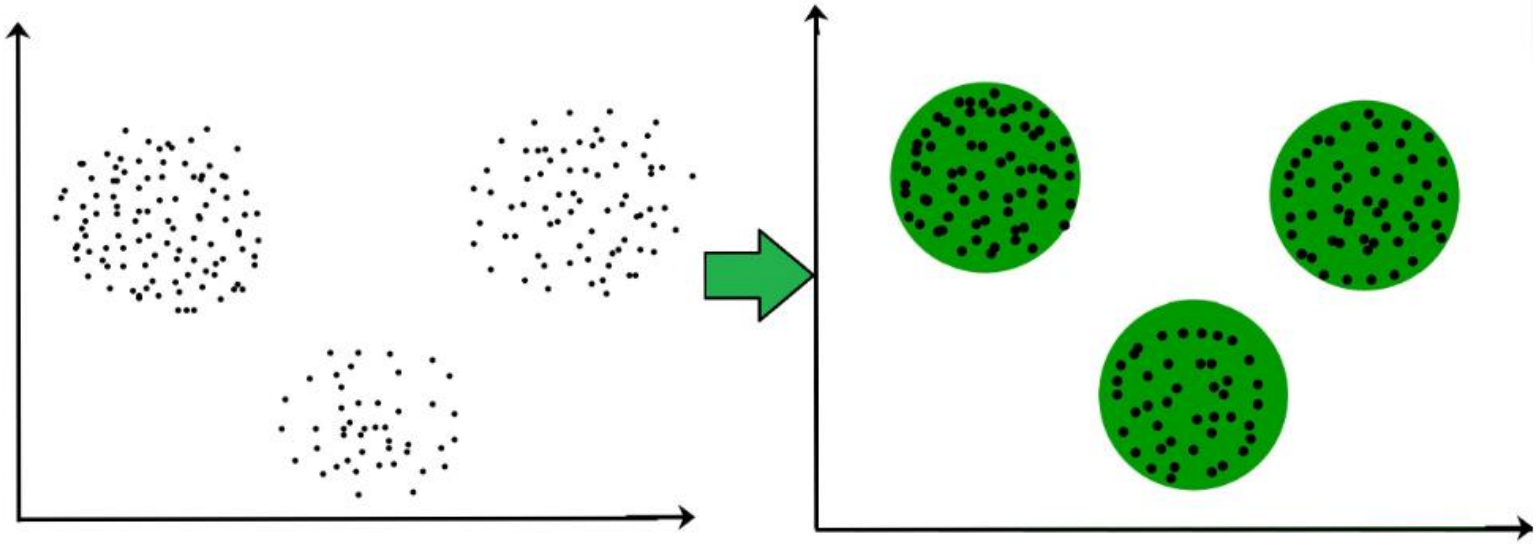
- clustering ⟶ grouping similar objects together into clusters

- it's an unsupervised task

from lab 1 ☺



- the goal of clustering:



2) *K*-means clustering

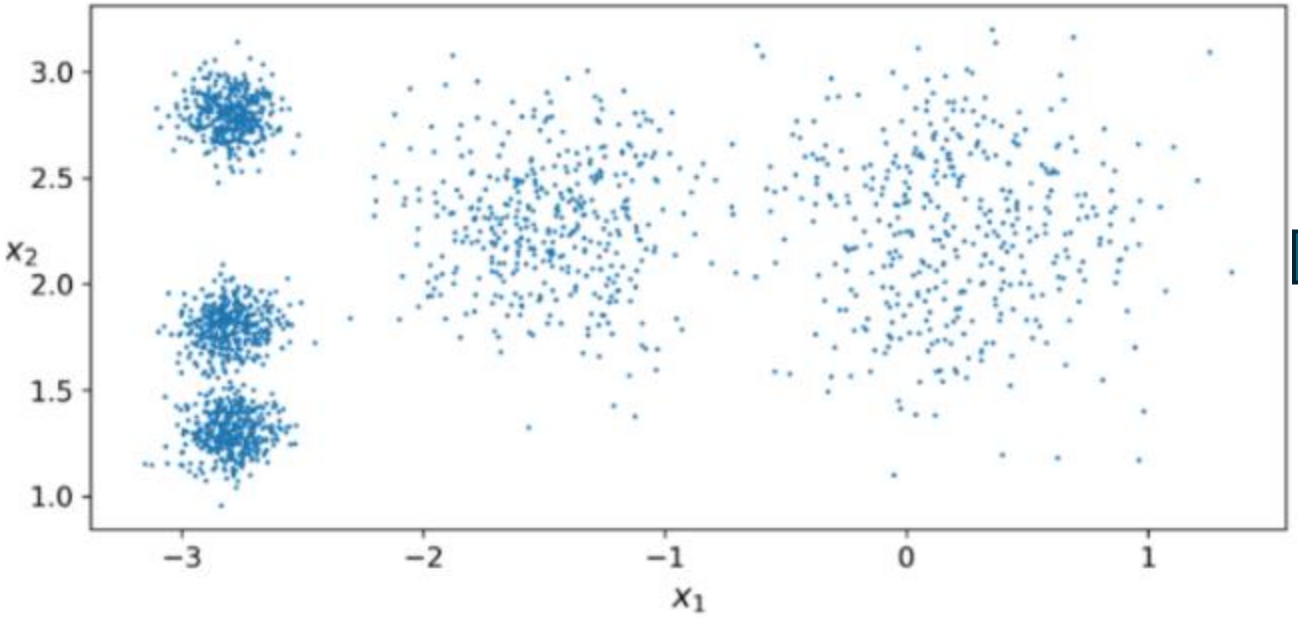
StatQuest: K-means clustering

1,6 mil. de vizionări • acum 6 ani

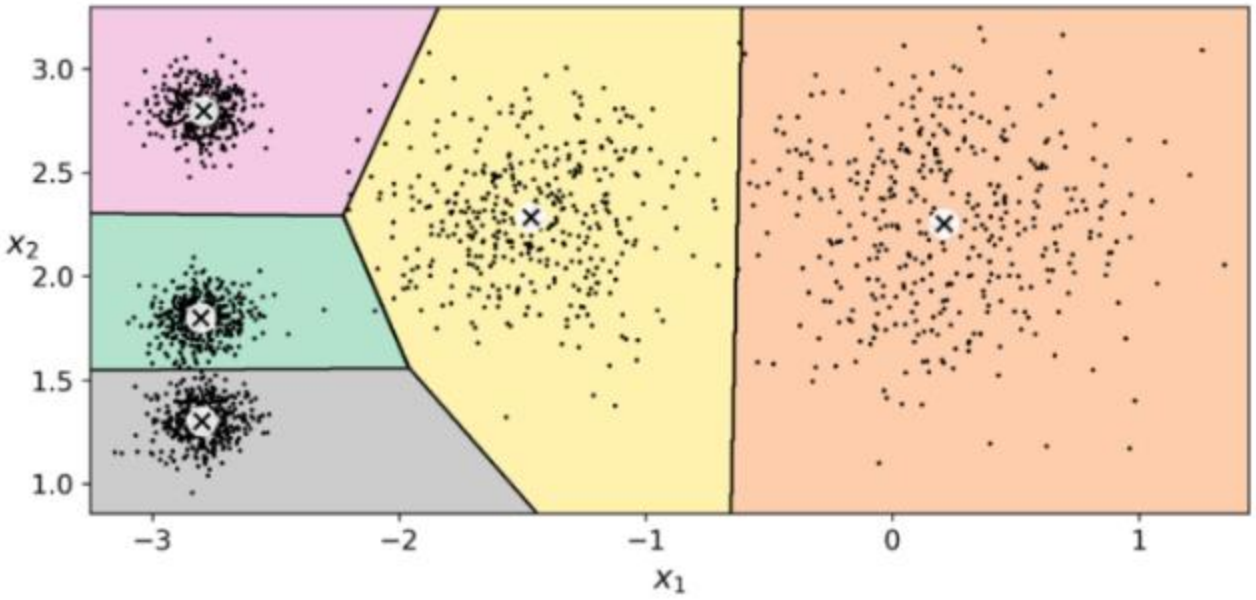
StatQuest with Josh Starmer

If you'd like to support StatQuest, please consider... Buying The StatQuest Illustrated Guide to Machine Learning!

Capitole: 6 Awesome song and introduction | The K-means clustering algorithm | How to pick a value for K (Ho...



- we can see here 5 blobs of points
- scope of *K*-means: find the center of each blob
- for a new point, we assign it to the closest blob/cluster



K-means steps

1. Choose the Number of Clusters (k)

- **Input**: Decide how many clusters (k) you want to partition your data into. This value can be based on prior knowledge, trial and error, or techniques like the elbow method.
- **Impact**: The chosen value of k significantly affects the clustering result, so it's essential to select it carefully.

2. Initialize Cluster Centroids

- **Random Initialization**: Randomly select k points from the data as the initial centroids (these are the "centers" of the clusters). These centroids serve as starting points for each cluster.

3. Assign Each Data Point to the Nearest Centroid (Cluster Assignment Step)

- For each data point in the dataset, calculate the **distance** to each of the k centroids. Typically, **Euclidean distance** is used, but other distance metrics can also be applied depending on the data and problem.
- **Assign the data point** to the cluster whose centroid is closest to it. This creates k clusters with each point belonging to the cluster with the nearest centroid.

4. Update the Centroids (Centroid Update Step)

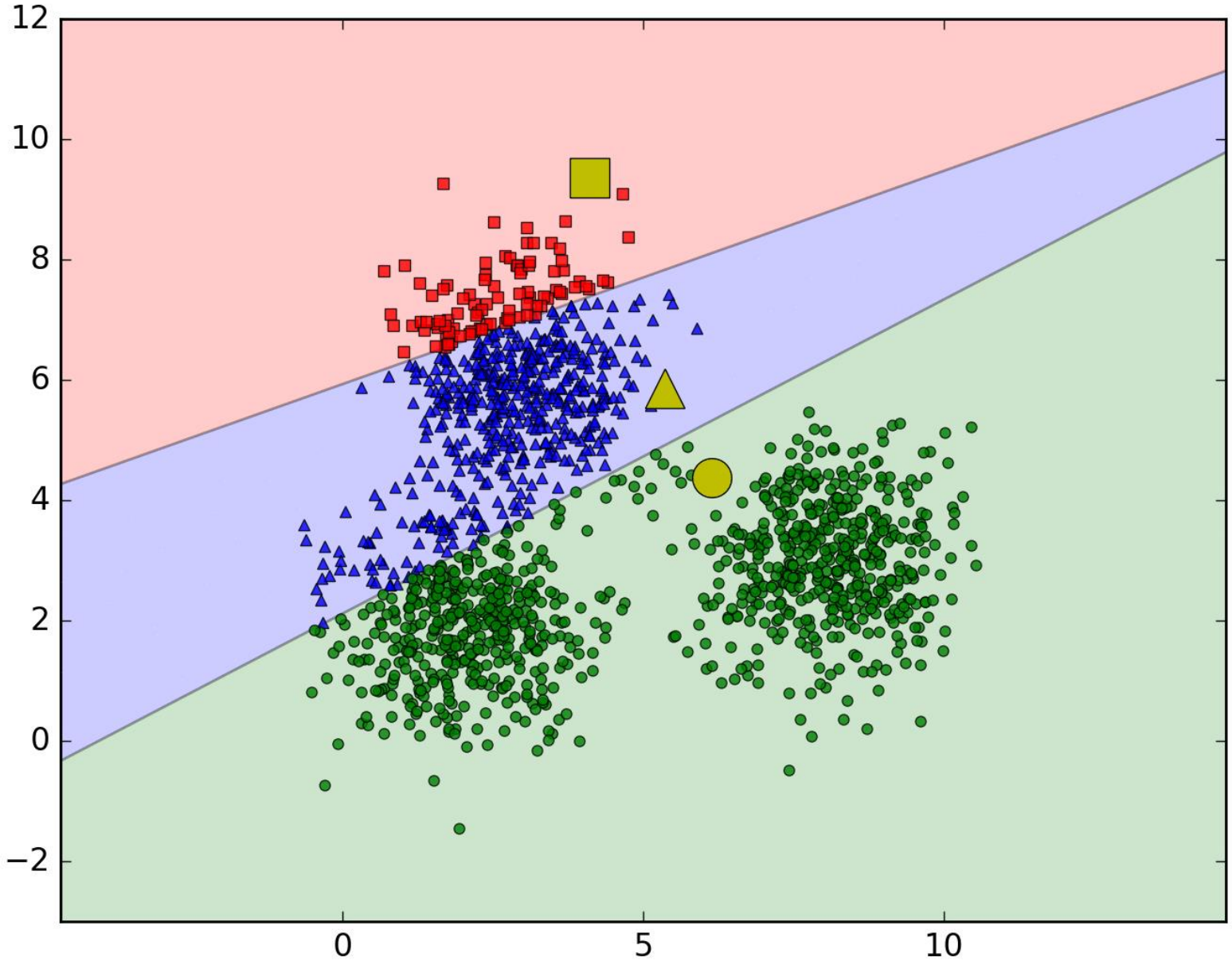
- **Recalculate the Centroid**: After assigning all points to clusters, recompute the centroid of each cluster. The new centroid is the **mean position of all the points in that cluster**:

$$\text{New Centroid} = \frac{1}{n}\sum_{i=1}^{n} x_i$$

  where x_i are the points in the cluster and n is the number of points in that cluster.
- **Effect**: This re-centering step updates each cluster's centroid to better represent the data points in that cluster.

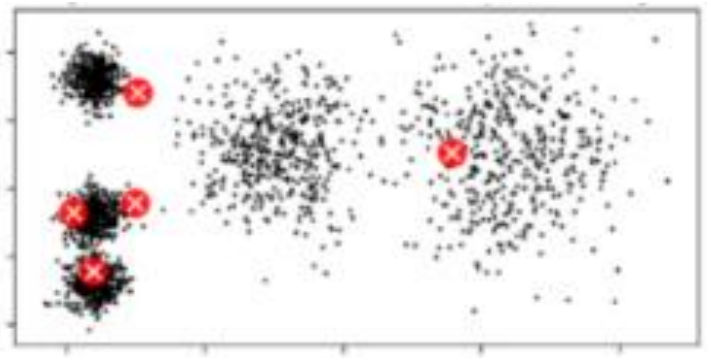
5. Repeat Steps 3 and 4 until Convergence

- Continue reassigning points to clusters and updating centroids until the centroids no longer change significantly (or meet another stopping criterion, like a maximum number of iterations).

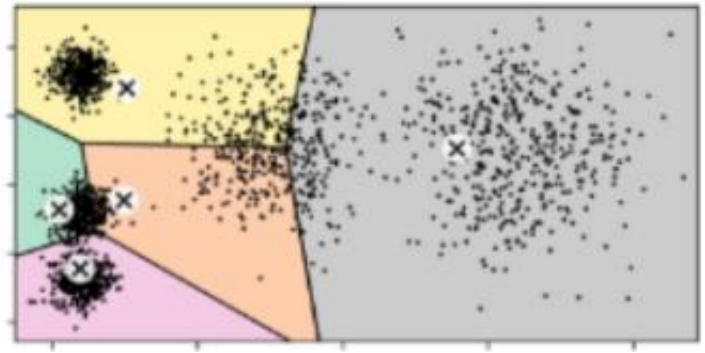


iteration 0: assign points to clusters

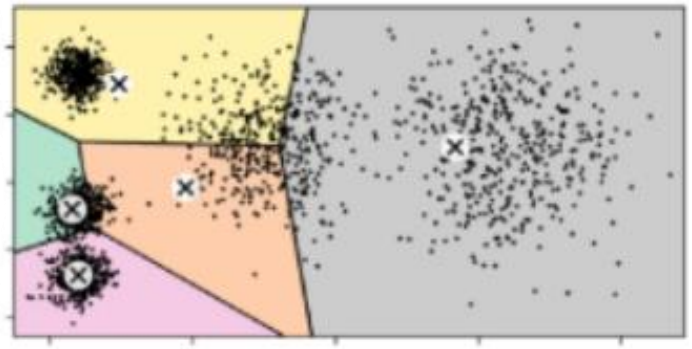
randomly initialize cluster centroids



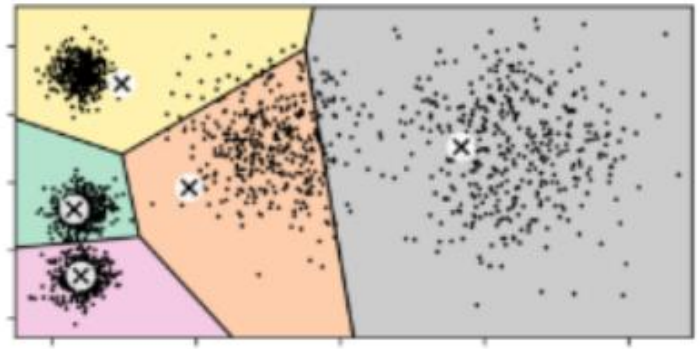
assign points to clusters



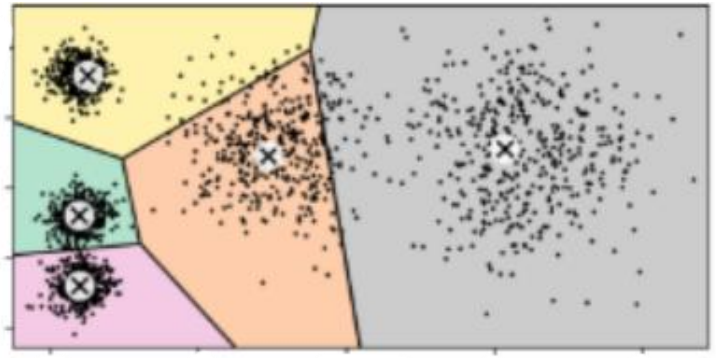
update the centroids



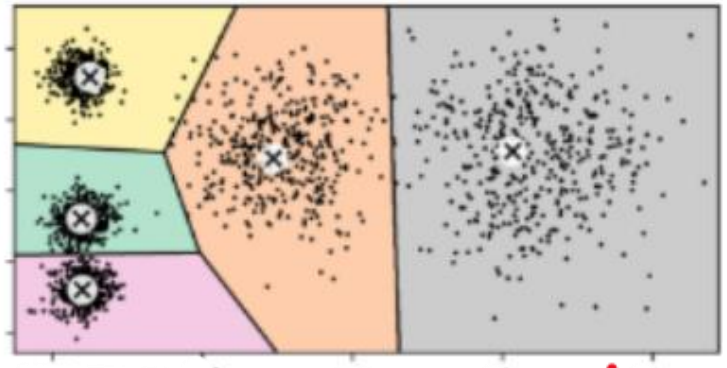
assign points to clusters



update the centroids



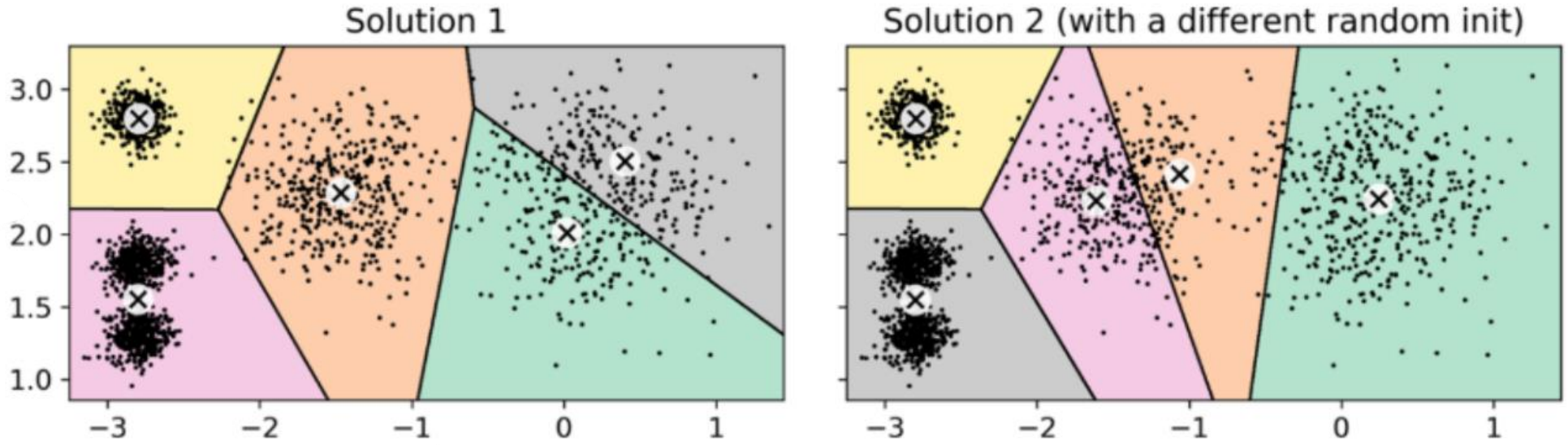
assign points to clusters



q.e.d.

NOTE:

2 bad solutions

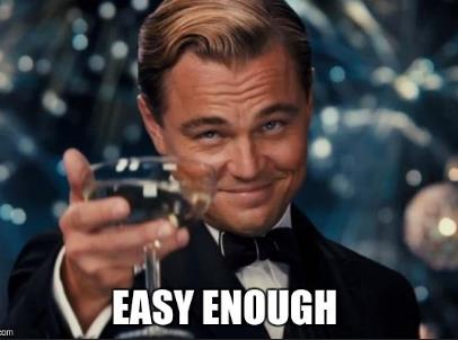


K-means is very sensitive to random initialization of the cluster centroids !

SOLUTIONS:

1) run *K*-means multiple times with different random initializations and keep the best solution (according to the metrics we will discuss in the following slides)

2) use *K*-means++ which introduces a smarter initialization of the cluster centroids



K-means++ Initialization Steps

Step 1: Choose the First Centroid Randomly

- **Action**: Select the first centroid (μ_1) uniformly at random from the data points.
- **Reason**: This provides a random starting point for the algorithm.

Step 2: Compute Distances to the Nearest Centroid

- **Action**: For each data point x_i, calculate the squared Euclidean distance $D(x_i)$ to the nearest centroid already chosen.

$$D(x_i) = \min_{\mu \in \{\mu_1\}} \|x_i - \mu\|^2$$

- **Reason**: This measures how far each point is from the closest centroid.

Step 3: Select the Next Centroid Based on Distance

- **Action**: Choose the next centroid (μ_2) from the data points, where the probability $P(x_i)$ of selecting point x_i is proportional to $D(x_i)$.

$$P(x_i) = \frac{D(x_i)}{\sum_j D(x_j)}$$

- **Reason**: Points that are farther from the existing centroids have a higher chance of being selected, promoting a spread of centroids.

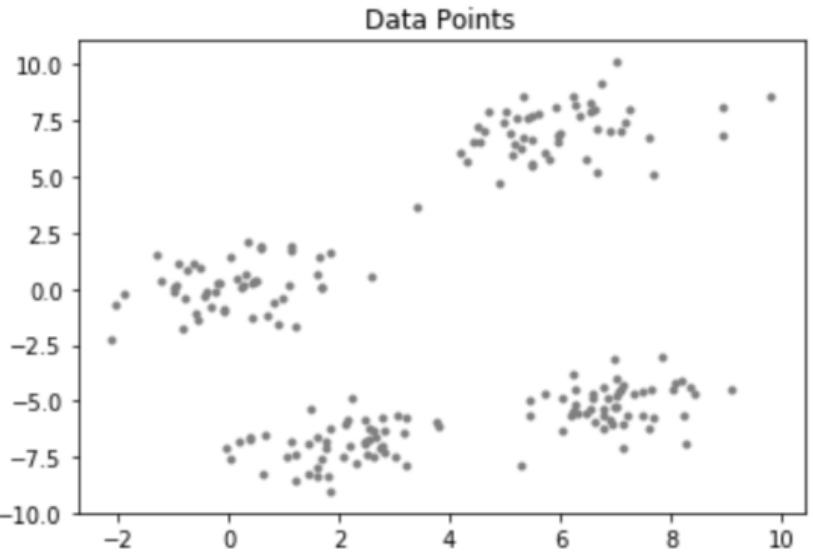
Step 4: Repeat Steps 2 and 3 Until k Centroids Are Chosen

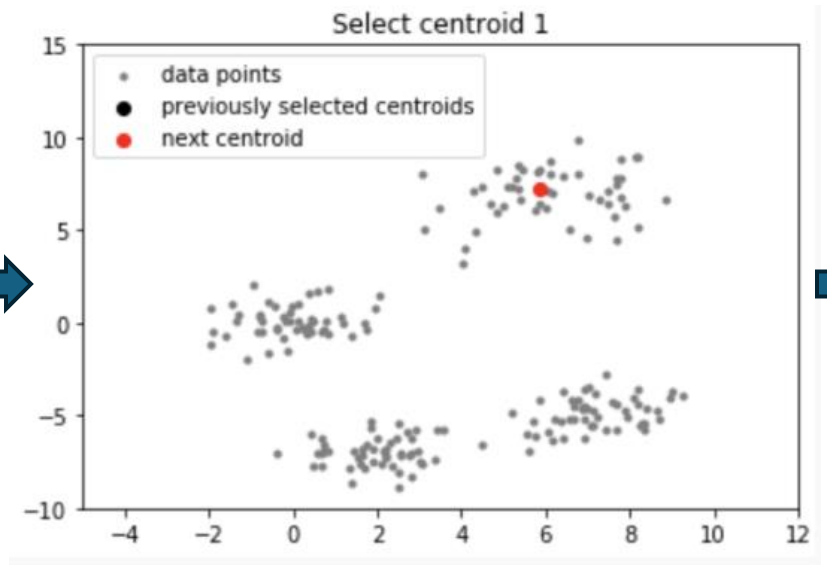
- **Action**: Update the set of centroids and recompute $D(x_i)$ for each data point using the nearest centroid among all selected centroids.

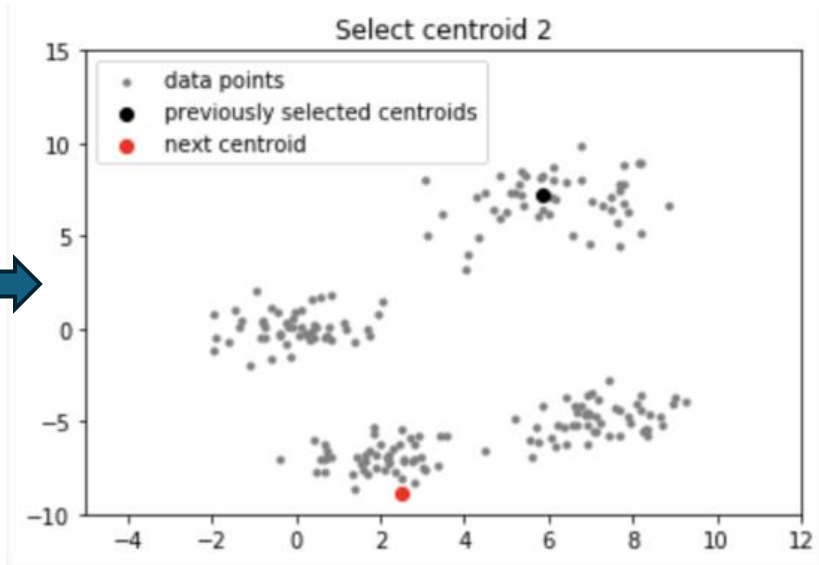
$$D(x_i) = \min_{\mu \in \{\mu_1, \mu_2, \ldots, \mu_m\}} \|x_i - \mu\|^2$$

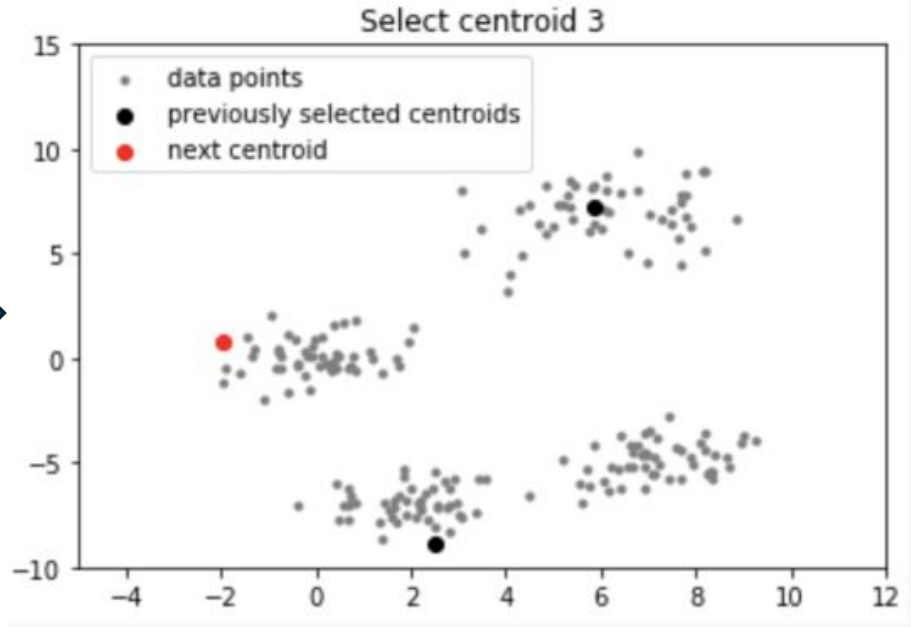
  - Where m is the number of centroids selected so far.

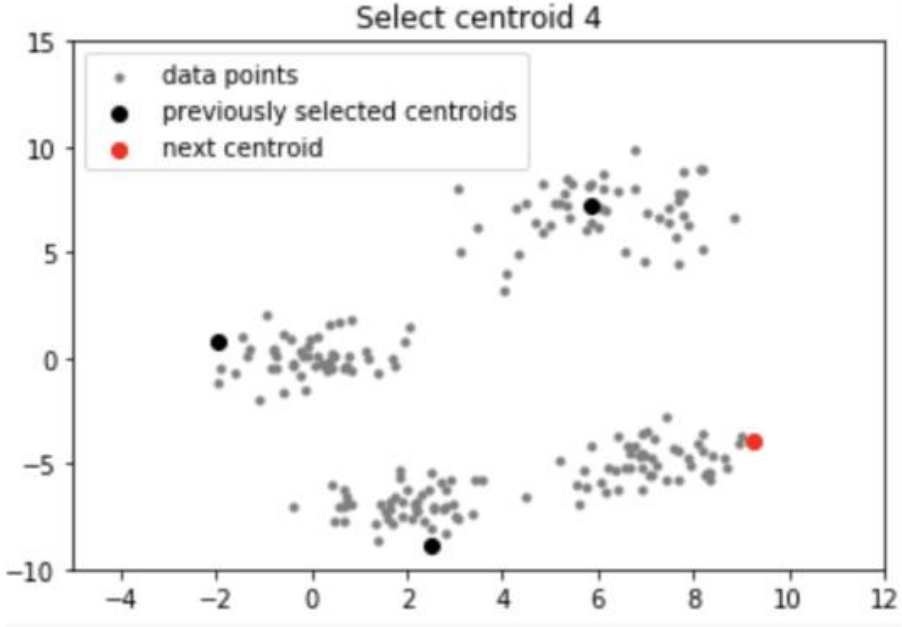
- **Reason**: Continually updating distances ensures that new centroids are chosen from regions that are far from existing centroids.


Data Points


Select centroid 1
data points
previously selected centroids
next centroid


Select centroid 2
data points
previously selected centroids
next centroid


Select centroid 3
data points
previously selected centroids
next centroid


Select centroid 4
data points
previously selected centroids
next centroid

How do we choose the optimal *k*?

We have 2 performance metrics:

1) **inertia**
2) **silhouette score**

1) **Inertia**: it's the sum of squared distances between each data point and its assigned cluster's centroid

$$\text{Inertia} = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

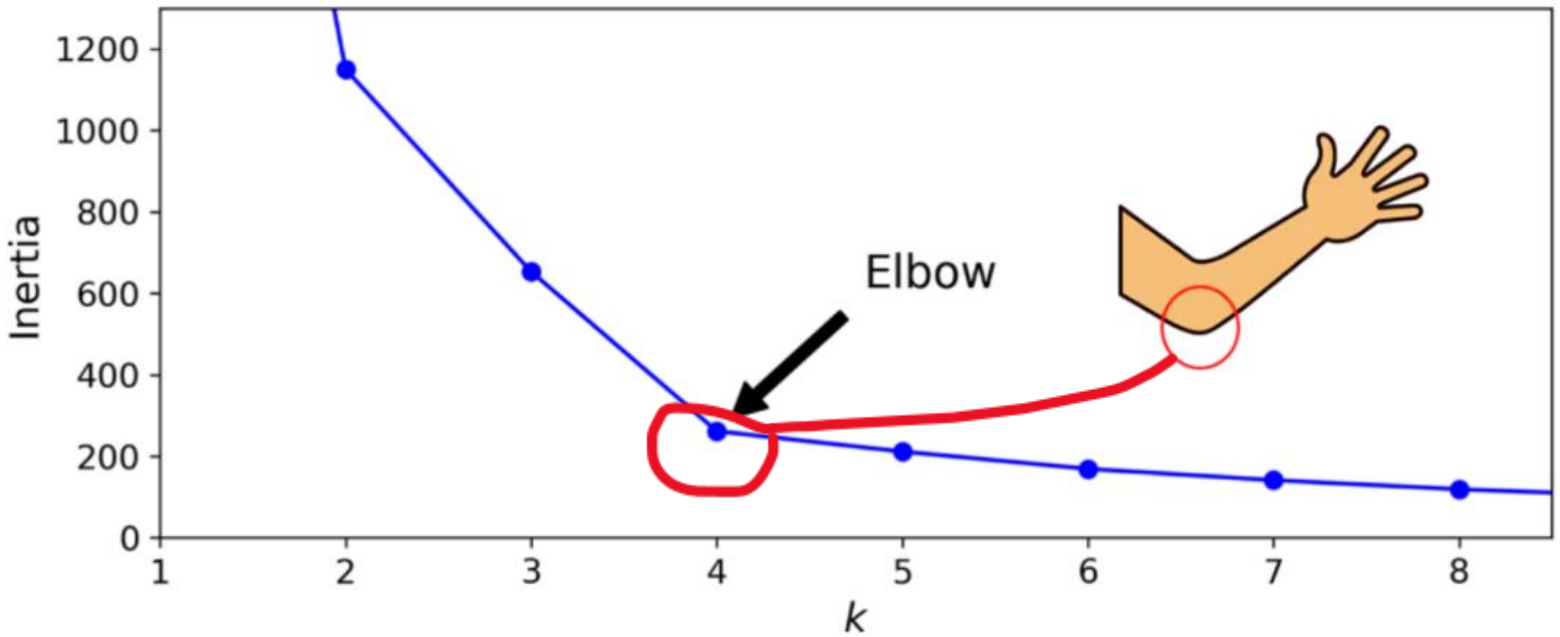
where:

- k is the number of clusters,
- C_i represents each cluster,
- x is a data point in cluster C_i,
- μ_i is the centroid of cluster C_i,
- $\|x - \mu_i\|^2$ is the squared Euclidean distance between point x and the centroid μ_i.

Intuition Behind Inertia:

- **Low Inertia**: A lower inertia value means that points are close to their respective centroids, indicating compact and well-defined clusters.
- **High Inertia**: A higher inertia value means points are spread out from their centroids, indicating loose or overlapping clusters.

Inertia generally **decreases** as k increases, because adding more clusters allows each centroid to move closer to a subset of data points. However, increasing k indefinitely may lead to overfitting, where each point might eventually become its own "cluster," yielding zero inertia.



We choose the optimal number of clusters *k* using the elbow method !

2) silhouette score:

The silhouette score for each data point measures two key factors:

1. **Cohesion**: How close each data point is to other points in the same cluster.
2. **Separation**: How far each data point is from points in the nearest neighboring cluster.

For each data point i:

- **a(i)**: The **mean intra-cluster distance**. This is the average distance from point i to all other points in the same cluster. It represents **cohesion** — lower values indicate that i is close to other points in its cluster.
- **b(i)**: The **mean nearest-cluster distance**. This is the average distance from point i to all points in the nearest neighboring cluster (i.e., the cluster with the lowest average distance to i that isn't its own cluster). This represents **separation** — higher values indicate that i is well-separated from other clusters.

The silhouette score for a single point i is calculated as:

$$\boxed{s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}}$$

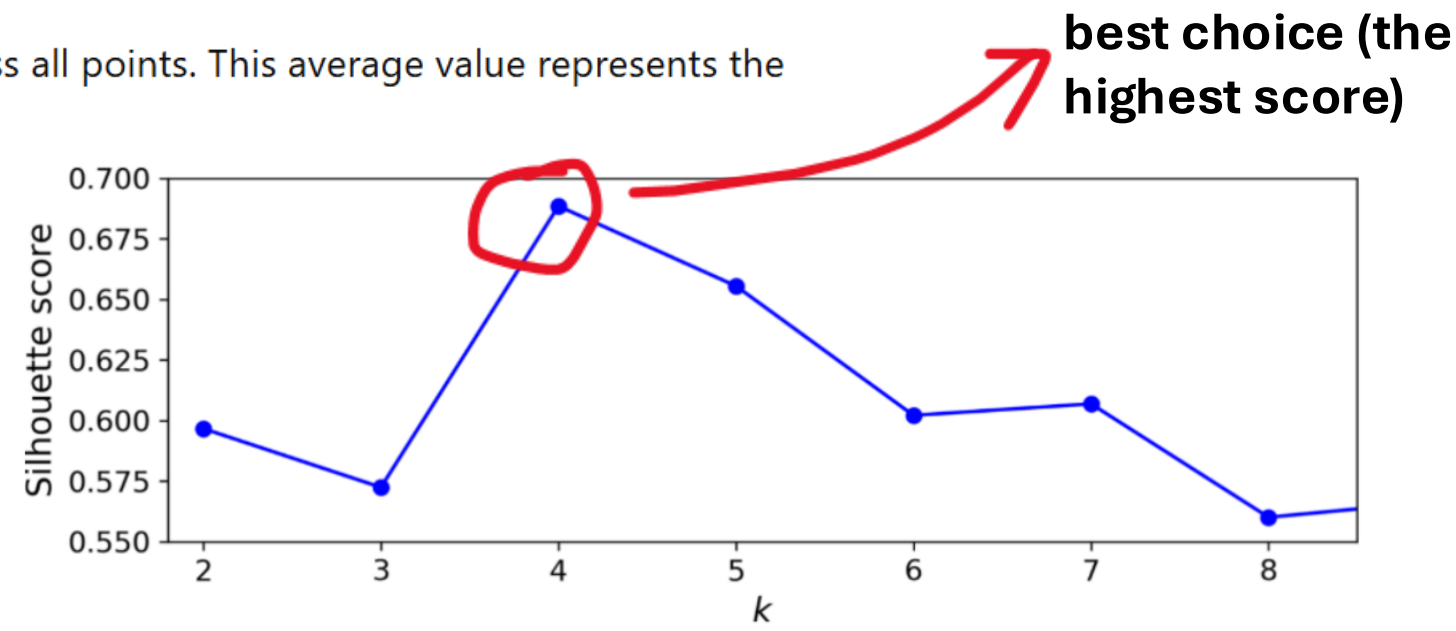
The silhouette score ranges from **-1 to 1**:

- **1**: The point is well-matched to its own cluster and poorly matched to neighboring clusters (ideal clustering).
- **0**: The point is on or very close to the boundary between two clusters.
- **-1**: The point is closer to a neighboring cluster than to its own cluster (indicating a poor clustering assignment).

Calculating the Overall Silhouette Score

To get the silhouette score for the entire clustering result:

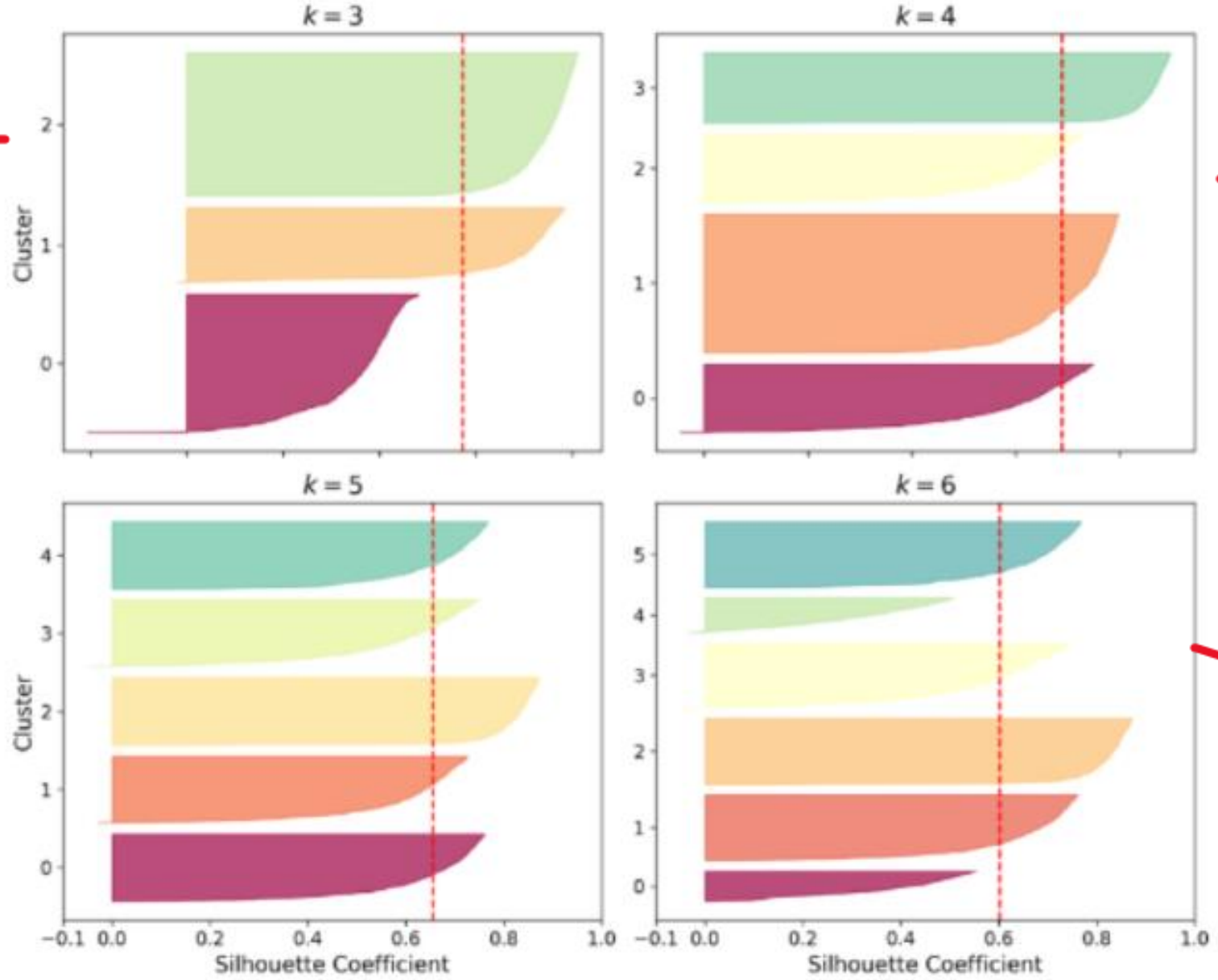
1. **Calculate** $s(i)$ for every data point in the dataset.
2. Take the **average of all silhouette scores** across all points. This average value represents the overall silhouette score of the clustering.



...or using a **silhouette diagram**...

bad option

good option



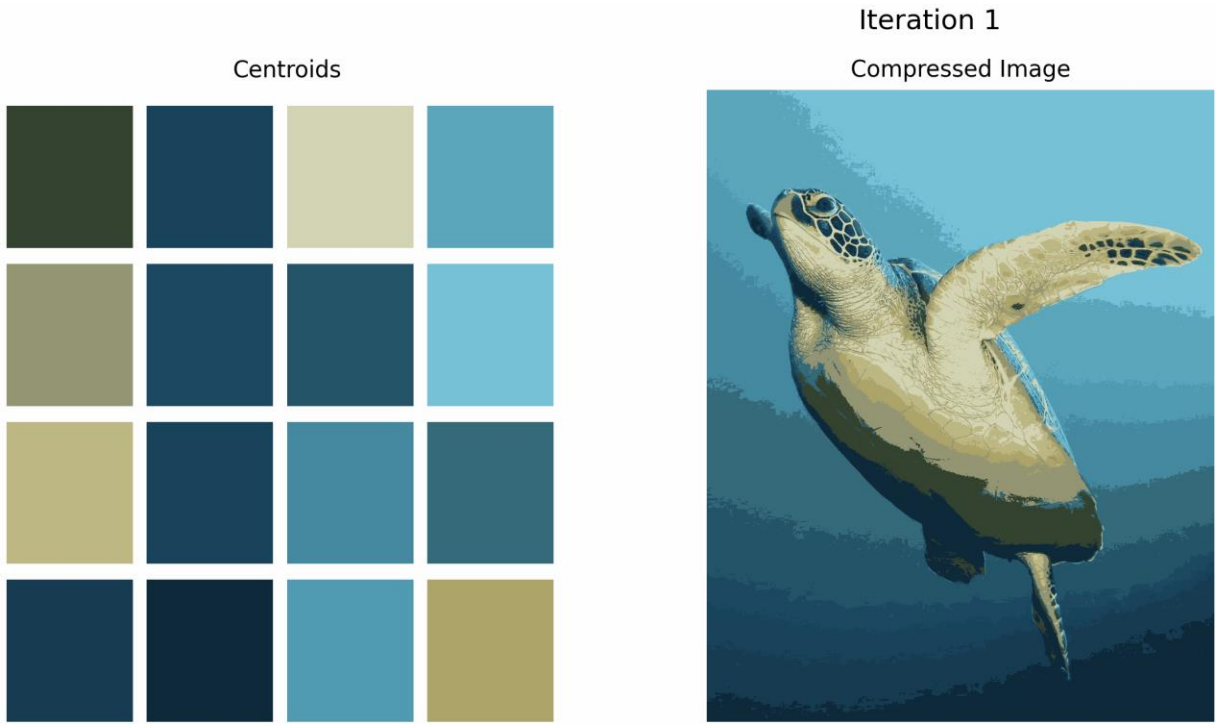
good option

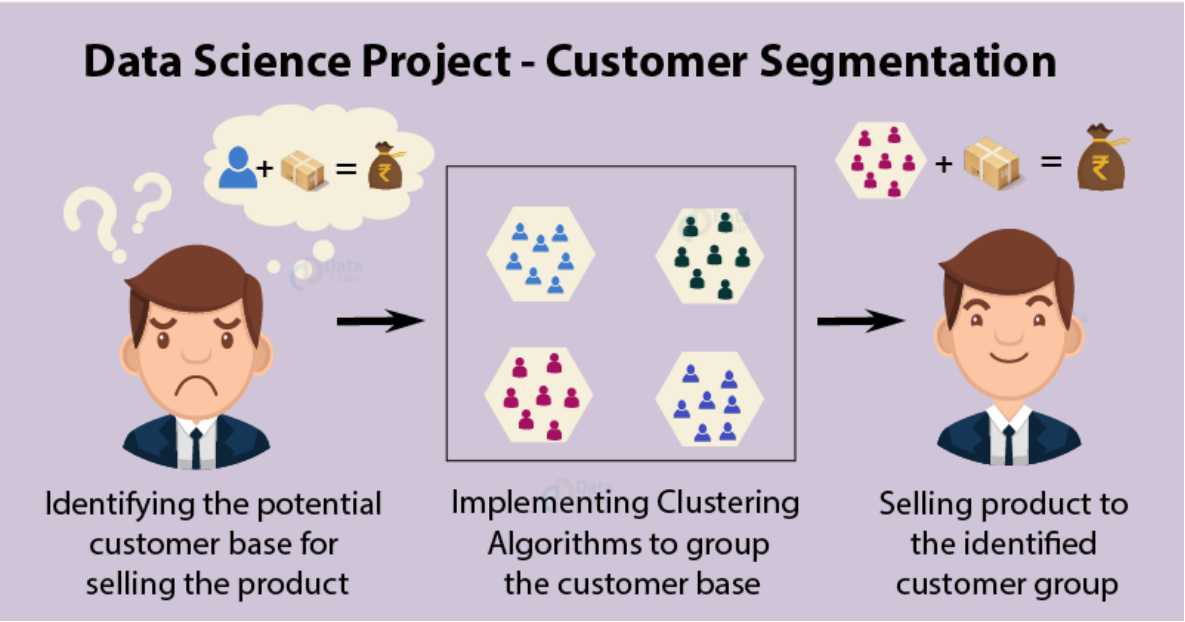
bad option

NOTES:

the shape's height indicates the number of instances the cluster contains, and its width represents the sorted silhouette coefficients of the instances in the cluster (wider is better);

the vertical dashed lines represent the silhouette score for each number of clusters;





K-Means Image Compression

1 **Convert:** Each pixel → 3D point $[R,\ G,\ B]$.
2 **Choose K:** Number of colors to keep (e.g., 16).
3 **Init Centroids:** Pick K random colors.
4 **Assign:** Each pixel → nearest centroid (color cluster).
5 **Update:** Each centroid = mean of its assigned pixels.
6 **Repeat:** Reassign + update until centroids stabilize.
7 **Reconstruct:** Replace each pixel with its centroid color → compressed image.
8 **Save:** Store palette (K colors) + pixel indices → smaller file.

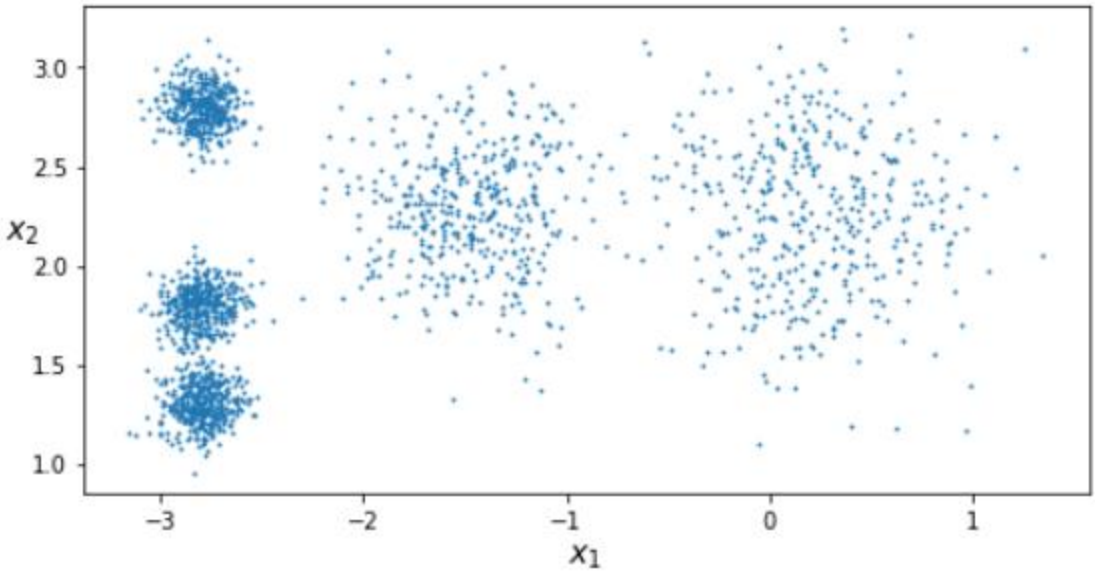
Customer Segmentation with Clustering

- **Goal:** Find groups of customers with similar behavior or preferences.
- **Input:** Data like age, income, spending habits, region, etc.
- **Method:** Apply clustering (e.g. **K-Means**)
- **Result:** Each cluster = a distinct customer segment (e.g. “budget buyers”, “premium spenders”).

Code part

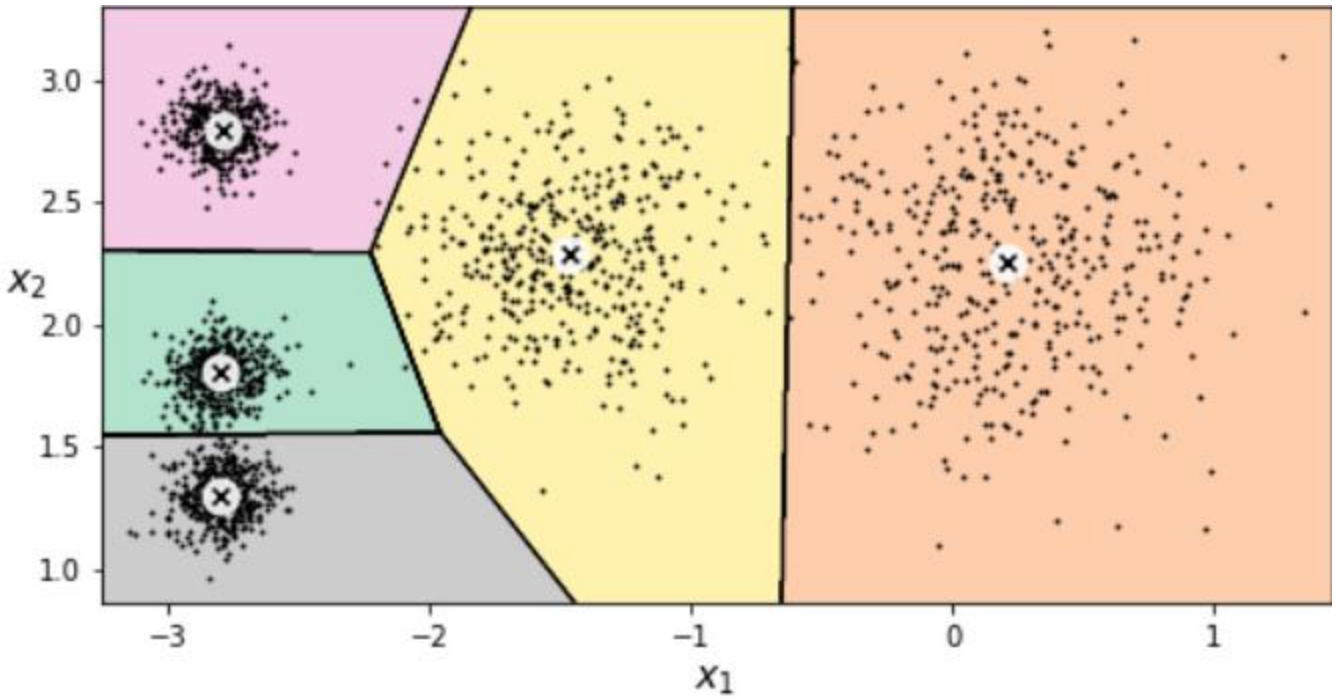
```python
from sklearn.datasets import make_blobs
import numpy as np
np.random.seed(42)

blob_centers = np.array(
    [[ 0.2,  2.3],
     [-1.5 ,  2.3],
     [-2.8,  1.8],
     [-2.8,  2.8],
     [-2.8,  1.3]])
blob_std = np.array([0.4, 0.3, 0.1, 0.1, 0.1])

X, y = make_blobs(n_samples=2000, centers=blob_centers,
                  cluster_std=blob_std, random_state=7)
```

```python
import matplotlib.pyplot as plt

def plot_clusters(X, y=None):
    plt.scatter(X[:, 0], X[:, 1], c=y, s=1)
    plt.xlabel("$x_1$", fontsize=14)
    plt.ylabel("$x_2$", fontsize=14, rotation=0)

plt.figure(figsize=(8, 4))
plot_clusters(X)
plt.show()
```



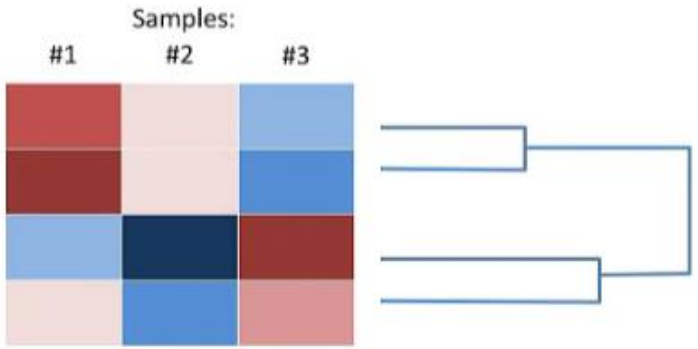
```python
from sklearn.cluster import KMeans

kmeans = KMeans(n_clusters=5, random_state=42)
y_pred = kmeans.fit_predict(X)
```

plot_decision_boundaries() function already done in lab☺



3) Agglomerative clustering

Hierarchical Clustering, Clearly Explained!!!!



StatQuest: Hierarchical Clustering

443 K de vizionări • acum 7 ani

StatQuest with Josh Starmer

If you'd like to support StatQuest, please consider... Buying The StatQuest Illustrated Guide to Machine Learning!

Capitole: 5 Intro | Example | Euclidean Distance | Manhattan Distance | Comparison

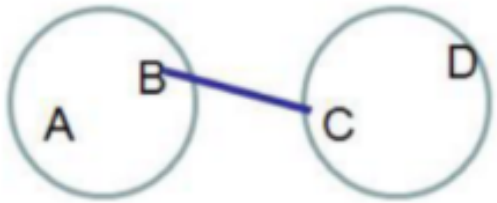
- overall idea: it starts by declaring each point its own cluster, and then merges the 2 most similar clusters, until some stopping criteria is satisfied

Step-by-Step Process of Agglomerative Clustering

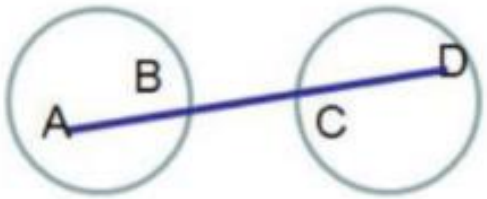
1. **Start with Each Data Point as Its Own Cluster**
   - Begin by treating each data point as a separate cluster.
   - If you have n data points, you start with n clusters.
2. **Compute the Distance Between Clusters**
   - Calculate the distance (or similarity) between each pair of clusters.
   - Common distance measures include **Euclidean distance**, **Manhattan distance**, or any metric suitable for your data.
3. **Merge the Two Closest Clusters**
   - Identify the two clusters that are closest to each other based on the distance metric.
   - Merge these two clusters into a single new cluster.
   - After each merge, you will have one less cluster.

4. **Update the Distances**
    - Recalculate the distances between the new cluster (formed by merging two clusters) and the remaining clusters.
    - This recalculation depends on the chosen **linkage method**:
        - **Single Linkage**: Distance between the two closest points in each cluster.
        - **Complete Linkage**: Distance between the two farthest points in each cluster.
        - **Average Linkage**: Average distance between points in the two clusters.
5. **Repeat Until Stopping Criteria is Met**
    - Continue finding the two closest clusters, merging them, and updating distances until:
        - Only one cluster remains (full hierarchy).
        - A specified number of clusters is reached.
        - The distance between clusters exceeds a threshold.
6. **Build the Dendrogram (Optional)**
    - As the algorithm progresses, it can be represented as a **dendrogram**, which is a tree-like diagram showing the order and distances at which clusters were merged.
    - The dendrogram helps visualize the hierarchy of clusters and allows for easy selection of an appropriate number of clusters by "cutting" it at a specific level.

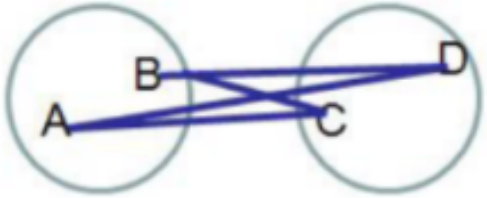
single linkage clustering $\rightarrow$ the distance between two groups is defined as the distance between the two closest members of each group;

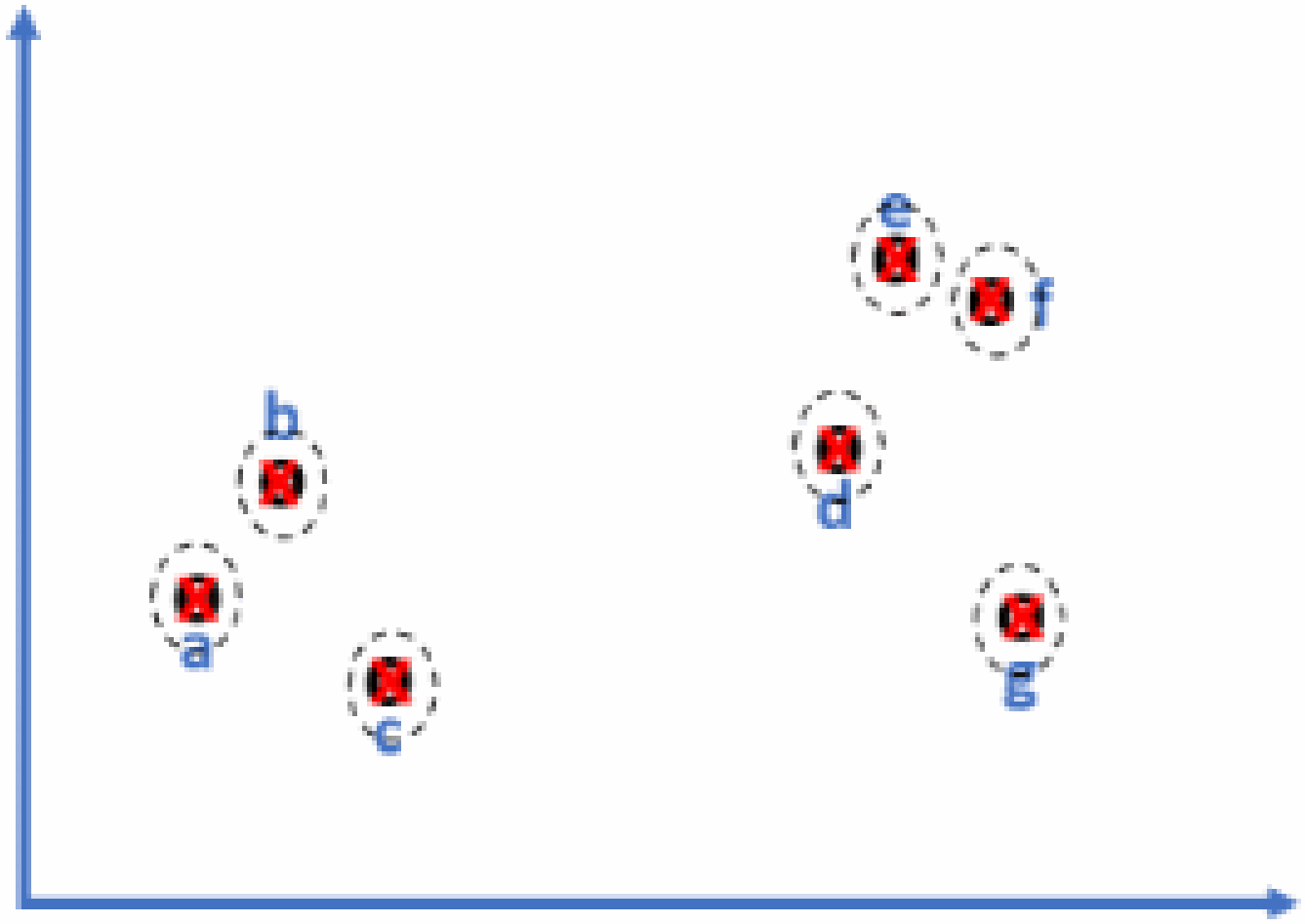


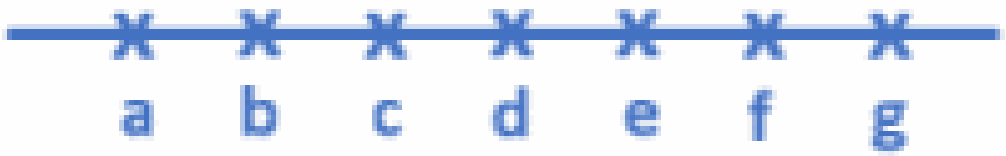
complete linkage clustering $\rightarrow$ the distance between two groups is defined as the distance between the two most distant pairs;



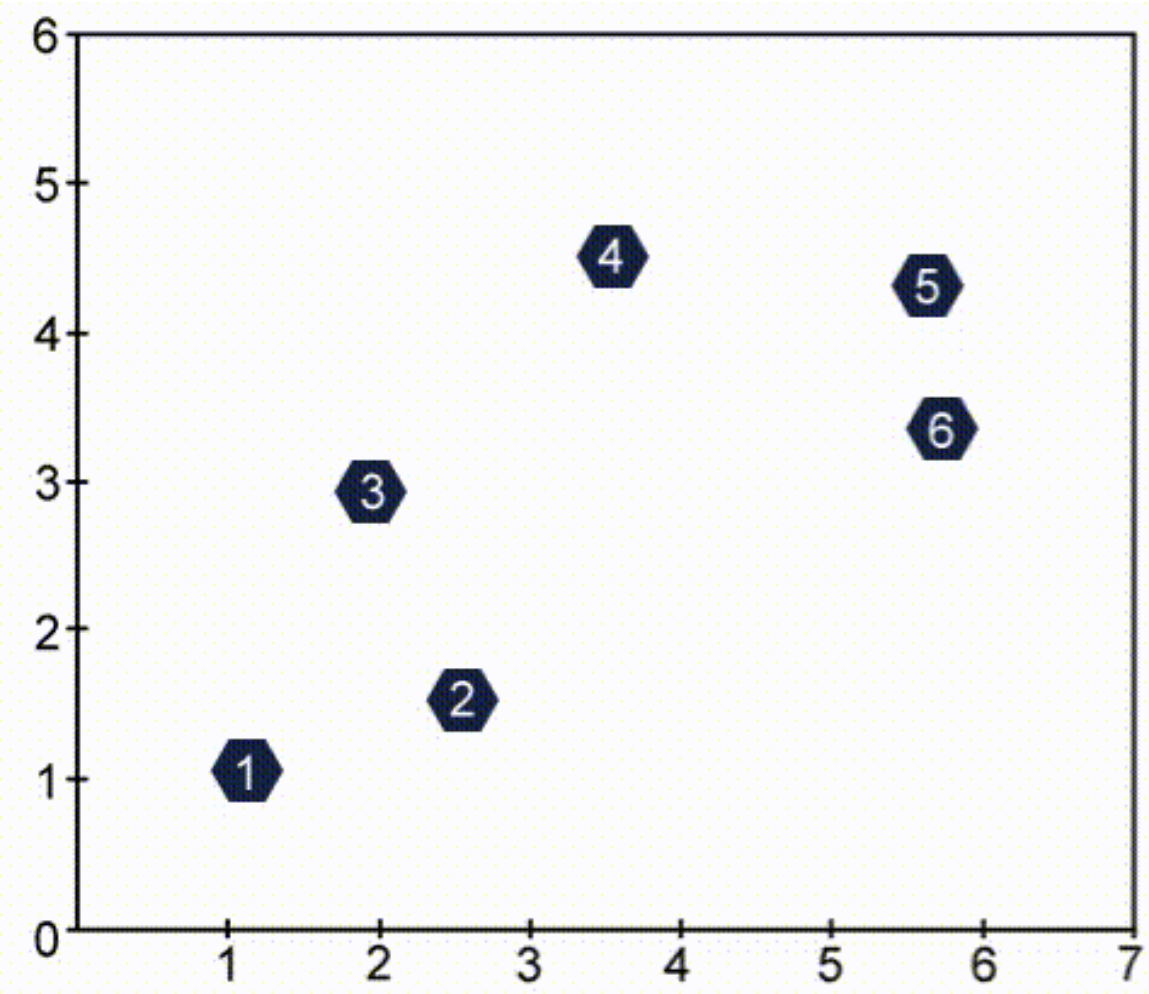
average link clustering $\rightarrow$ measures the average distance between all pairs;

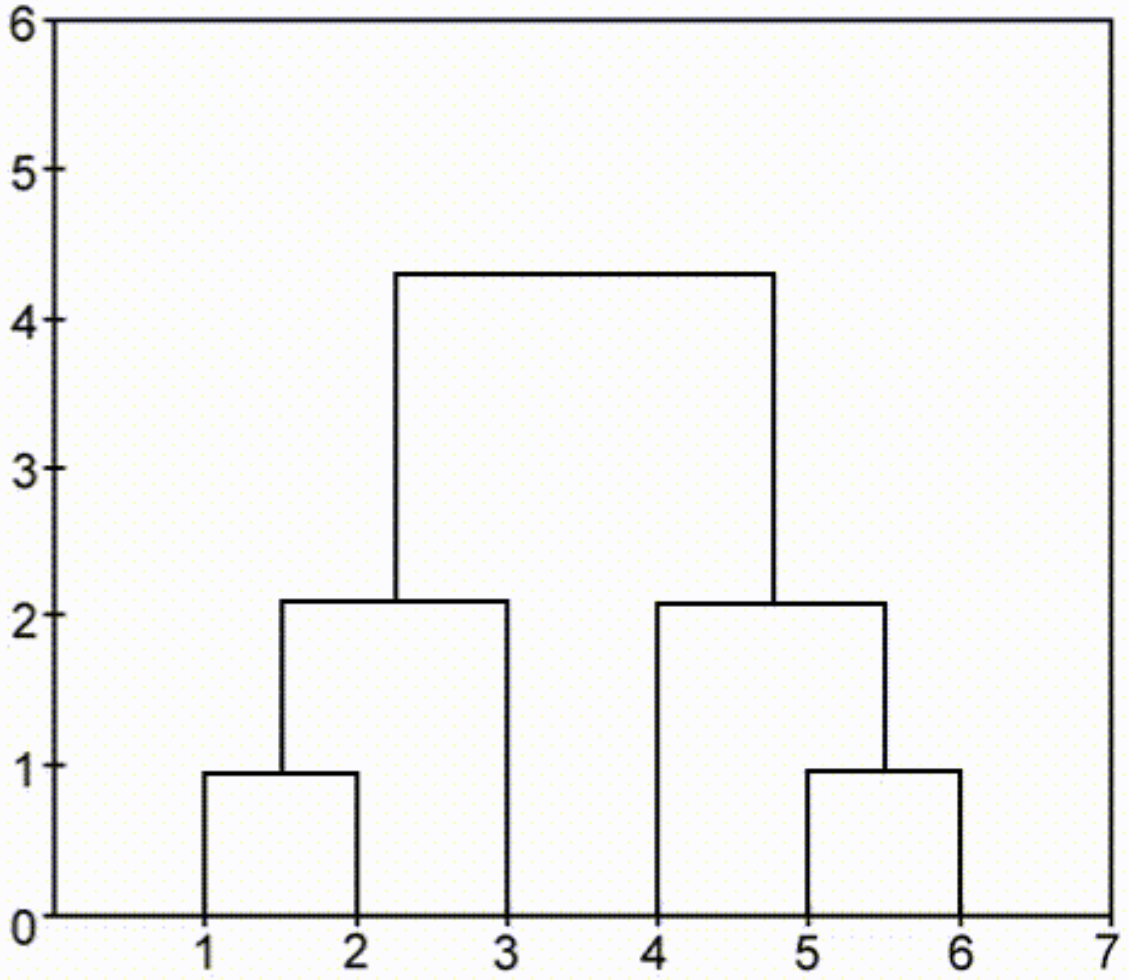






Dendrogram

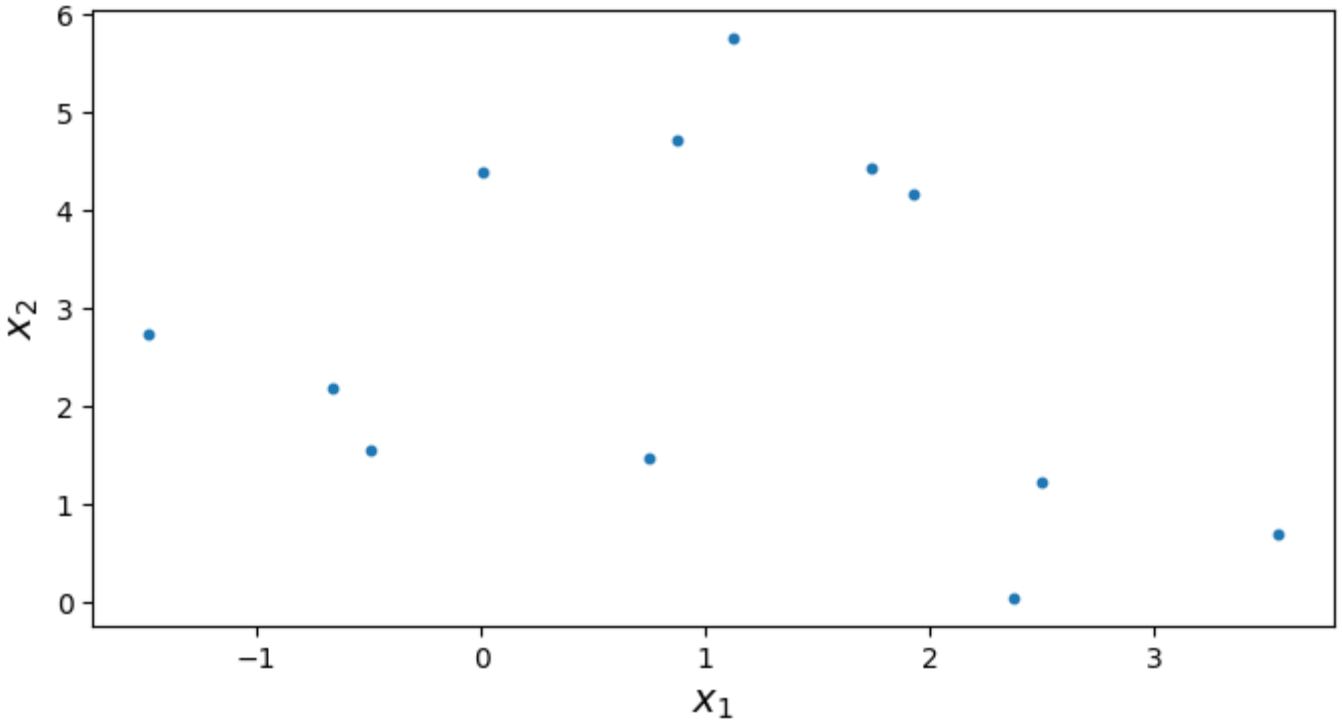




Dendrogram

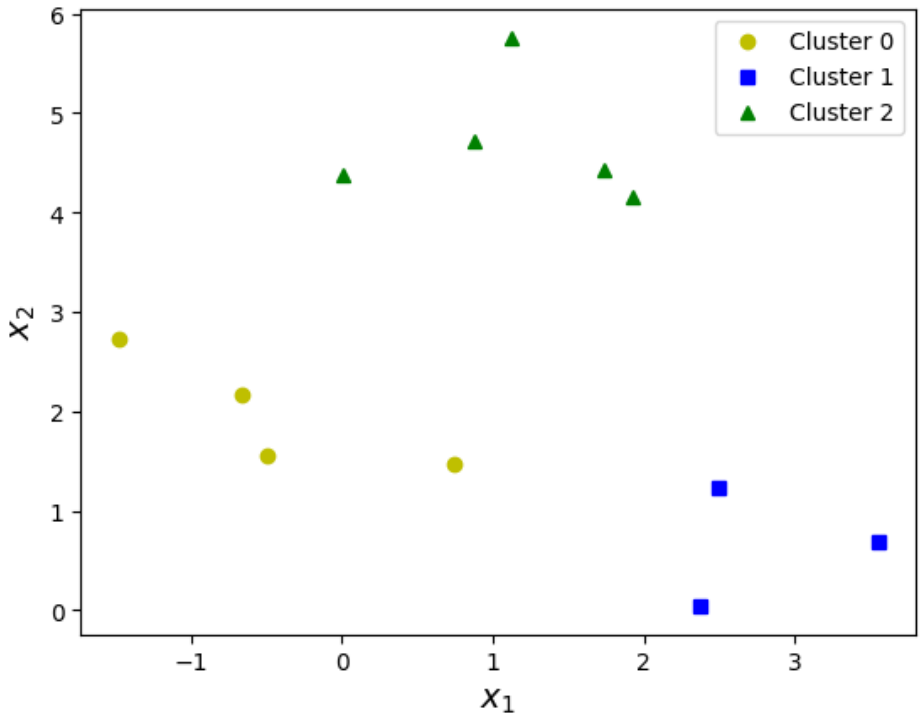
Code part

```python
import matplotlib.pyplot as plt
from sklearn.cluster import AgglomerativeClustering
X, y = make_blobs(random_state=0, n_samples=12)
```

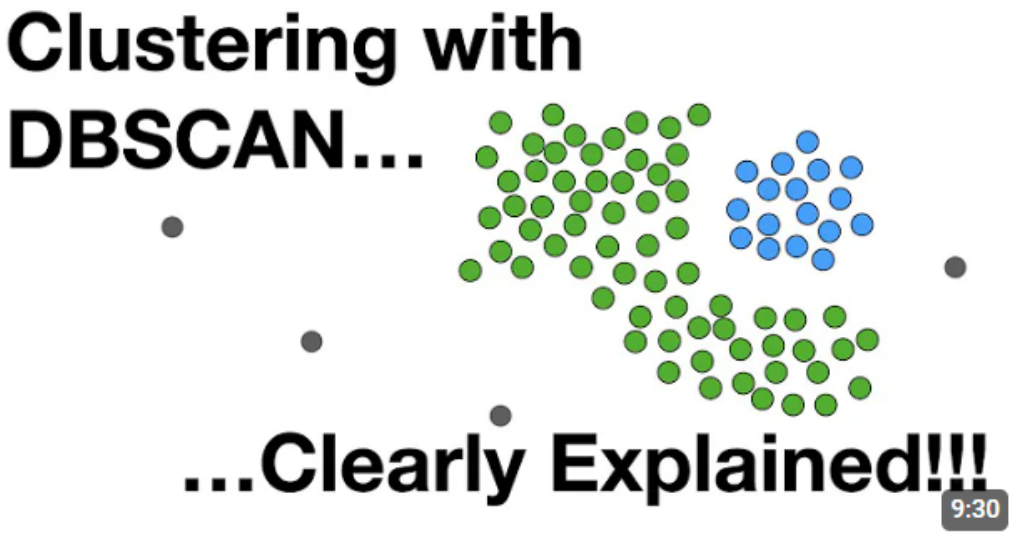


```python
agg = AgglomerativeClustering(n_clusters=3, linkage="average")
clusters = agg.fit_predict(X)
```

```python
plt.plot(X[clusters==0, 0], X[clusters==0, 1], "yo")
plt.plot(X[clusters==1, 0], X[clusters==1, 1], "bs")
plt.plot(X[clusters==2, 0], X[clusters==2, 1], "g^")
plt.legend(["Cluster 0", "Cluster 1", "Cluster 2"], loc="best")
plt.xlabel("$x_1$", fontsize=14)
plt.ylabel("$x_2$", fontsize=14)
plt.show()
```



4) DBSCAN
(density-based spatial clustering of applications with noise)



Clustering with DBSCAN, Clearly Explained!!!

317 K de vizionări • acum 2 ani

StatQuest with Josh Starmer

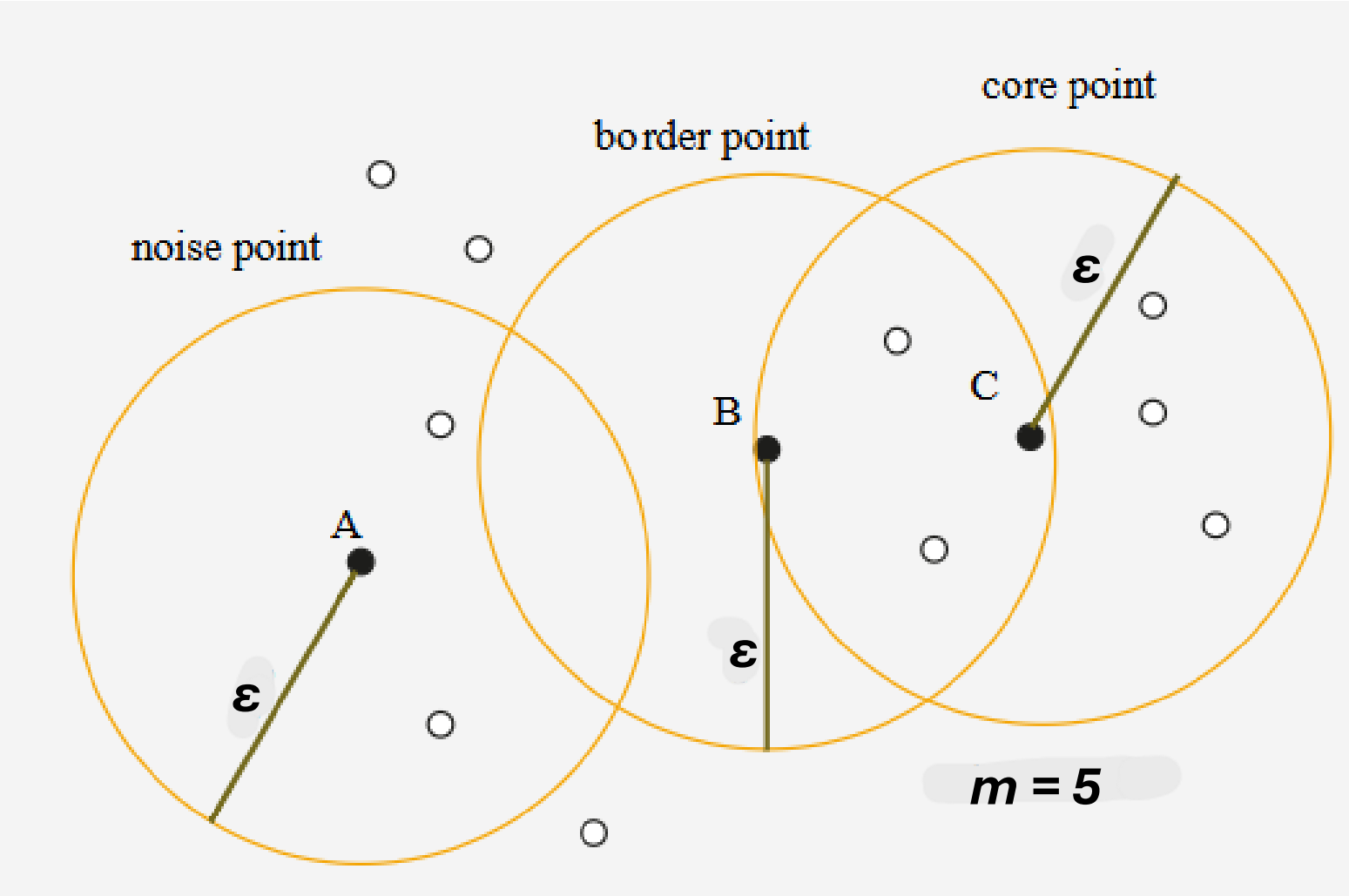
DBSCAN is a super useful clustering algorithm that can handle nested clusters with ease. This StatQuest shows you exactly how it ...

Capitole: 4 Awesome song and introduction | The problems solved by DBSCAN | How DBSCAN works | How...

- unlike *K*-means, DBSCAN doesn't require specifying the number of clusters beforehand and can identify clusters of arbitrary shapes, making it effective for tasks where the data contains noise or irregular clusters

- DBSCAN relies on 2 parameters:
  - ***ε***: the maximum distance between 2 points for them to be considered part of the same cluster.
  - ***m***: the minimum number of points required within a distance of ***ε*** for a point to be considered a core point.

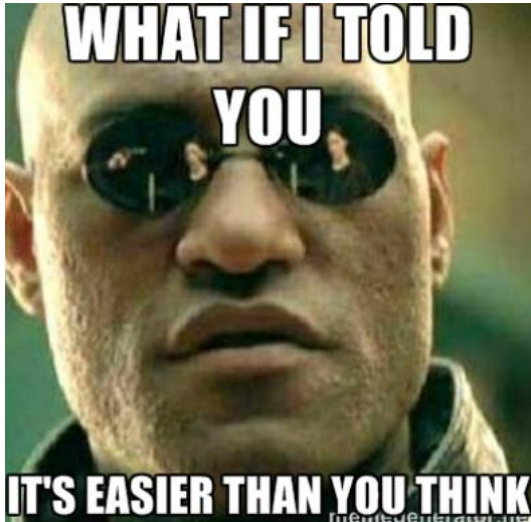
- based on these parameters, DBSCAN classifies each point as:
  - **core point**: a point with at least ***m*** points within its ***ε*** radius.
  - **border point**: a point that isn't a core point but is within ***ε*** of a core point.
  - **noise point**: a point that isn't a core point and isn't within ***ε*** of any core point.

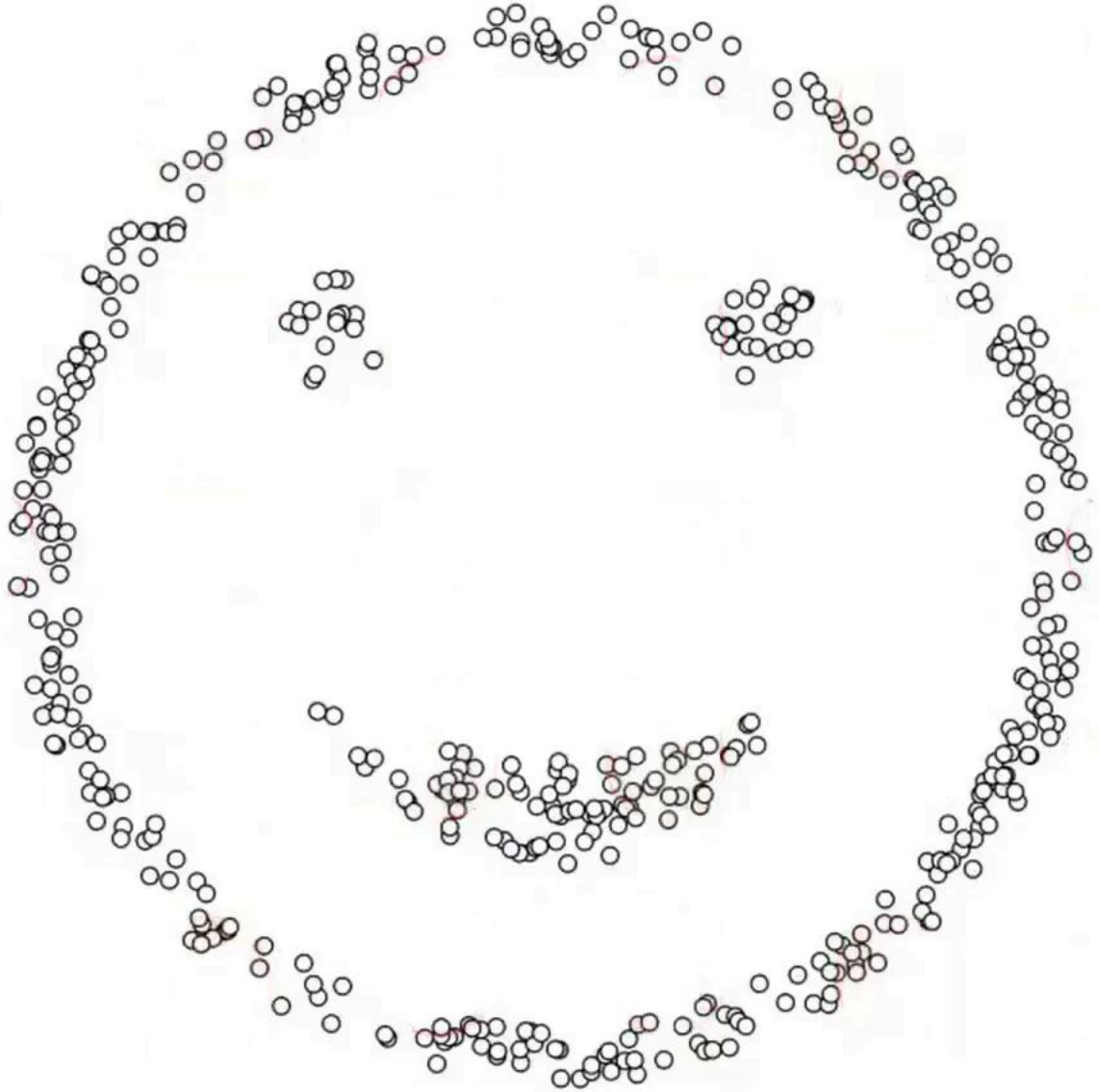

core point
border point
noise point
ε
ε
ε
A
B
C
m = 5

Don't worry about it ☺

Steps of the DBSCAN Algorithm

1. **Label All Points as Unvisited:**
   - Start with all points in the dataset labeled as "unvisited."
2. **Pick an Unvisited Point:**
   - Randomly select an unvisited point and label it as "visited."
3. **Check if the Point is a Core Point:**
   - Calculate the **eps-neighborhood** of the point (i.e., find all points within a distance ε from the chosen point).
   - If the point has at least m points within its **eps-neighborhood**, it is a core point and will be the starting point of a new cluster.
4. **Expand the Cluster from the Core Point:**
   - Create a new cluster with the core point as its initial member.
   - **For each point in the eps-neighborhood:**
     - If the point is unvisited, mark it as visited.
     - If it has at least m points within its **eps-neighborhood** (i.e., it's also a core point), add all its **eps-neighbors** to the list of points to be examined (this step grows the cluster).
     - If it doesn't have enough points to be a core point, it's a border point but is still added to the current cluster.
5. **Repeat the Expansion Until No More Points Can be Added:**
   - Continue adding reachable points to the cluster until no more points within ε can be added to the current cluster.
6. **Move to the Next Unvisited Point:**
   - If the selected point does not have enough neighbors (fewer than m within ε), label it as noise (temporarily).
   - Select another unvisited point and repeat steps 3-5 until all points have been visited and either assigned to a cluster or labeled as noise.
7. **Final Clustering Output:**
   - Points assigned to clusters represent the final clusters.
   - Points that remain labeled as noise are considered outliers or noise and do not belong to any cluster.





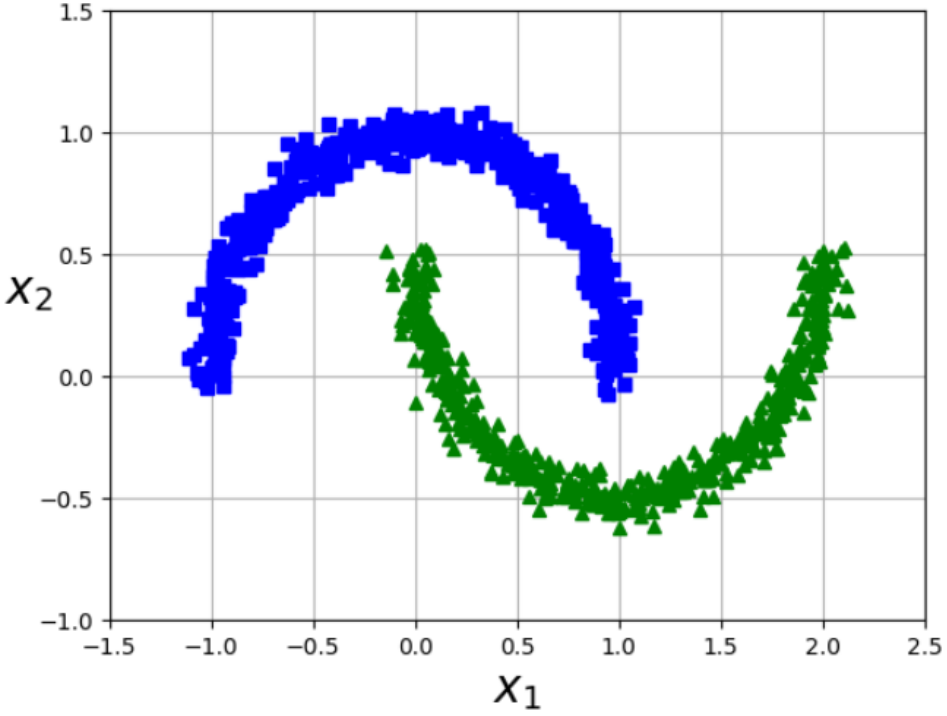
$\varepsilon = 1.10$
$m = 4$

Code part

```python
from sklearn.datasets import make_moons

X, y = make_moons(n_samples=1000, noise=0.05, random_state=42)
```

```python
def plot_dataset(X, y, axes):
    plt.plot(X[:, 0][y==0], X[:, 1][y==0], "bs")
    plt.plot(X[:, 0][y==1], X[:, 1][y==1], "g^")
    plt.axis(axes)
    plt.grid(True, which='both')
    plt.xlabel(r"$x_1$", fontsize=20)
    plt.ylabel(r"$x_2$", fontsize=20, rotation=0)

plot_dataset(X, y, [-1.5, 2.5, -1, 1.5])
plt.show()
```

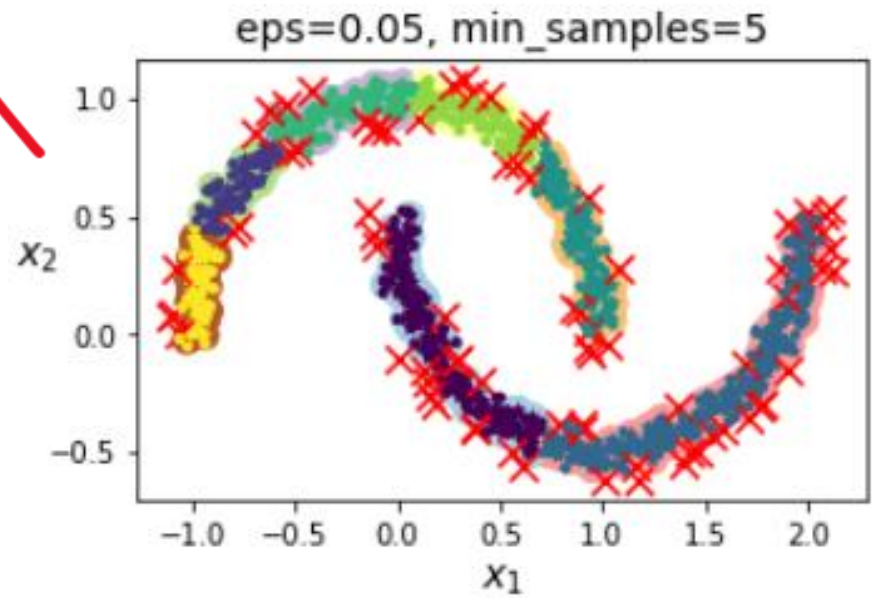
function from lab 3 ☺



```python
from sklearn.cluster import DBSCAN

dbscan = DBSCAN(eps=0.05, min_samples=5)
dbscan.fit(X)
```

plot_dbscan() function already done in lab☺

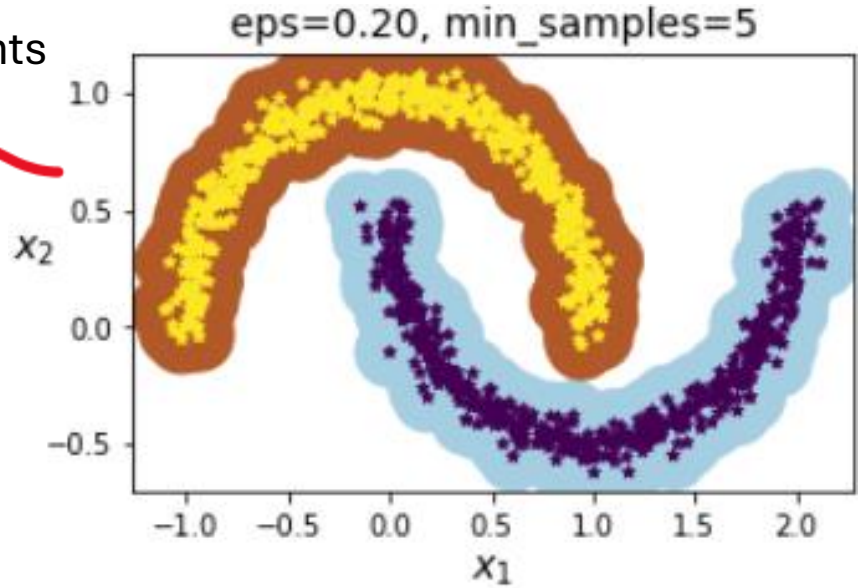
- 7 clusters
- noise points marked with red cross




```python
from sklearn.cluster import DBSCAN

dbscan2 = DBSCAN(eps=0.2, min_samples=5)
dbscan2.fit(X)
```

plot_dbscan() function already done in lab☺

- 2 clusters
- no noise points

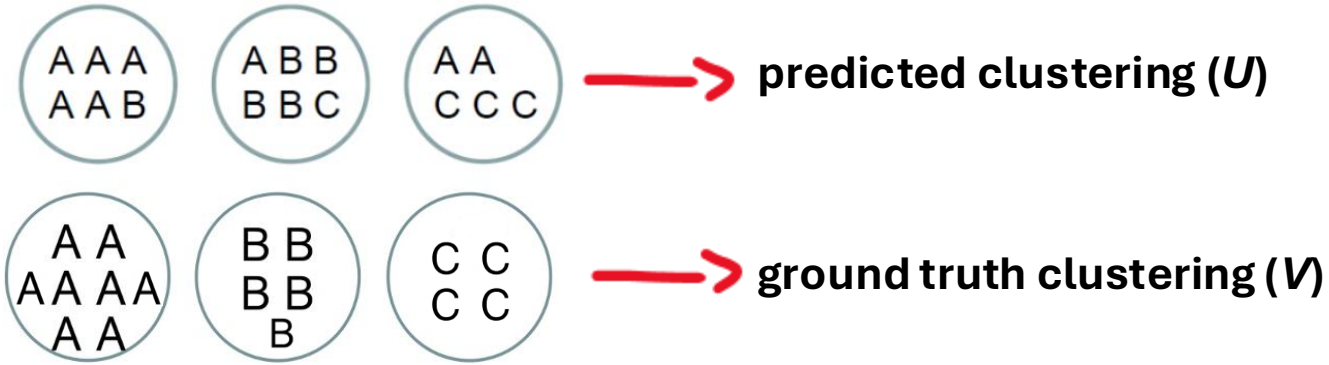


4) Evaluating clustering methods

2 methods:

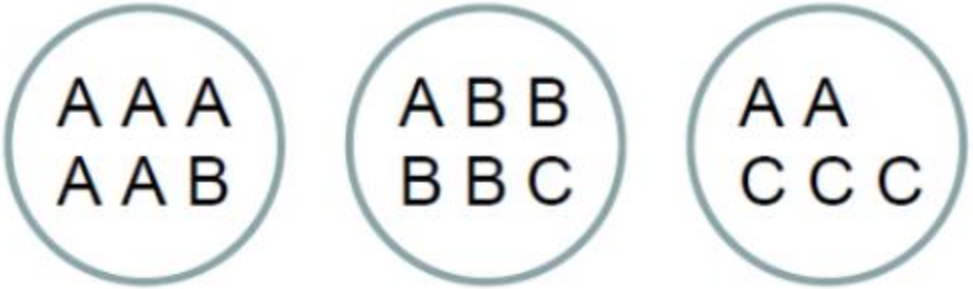
1) **ARI** (adjusted Rand index)
2) **NMI** (normalized mutual information)

1) **ARI** (adjusted Rand index)

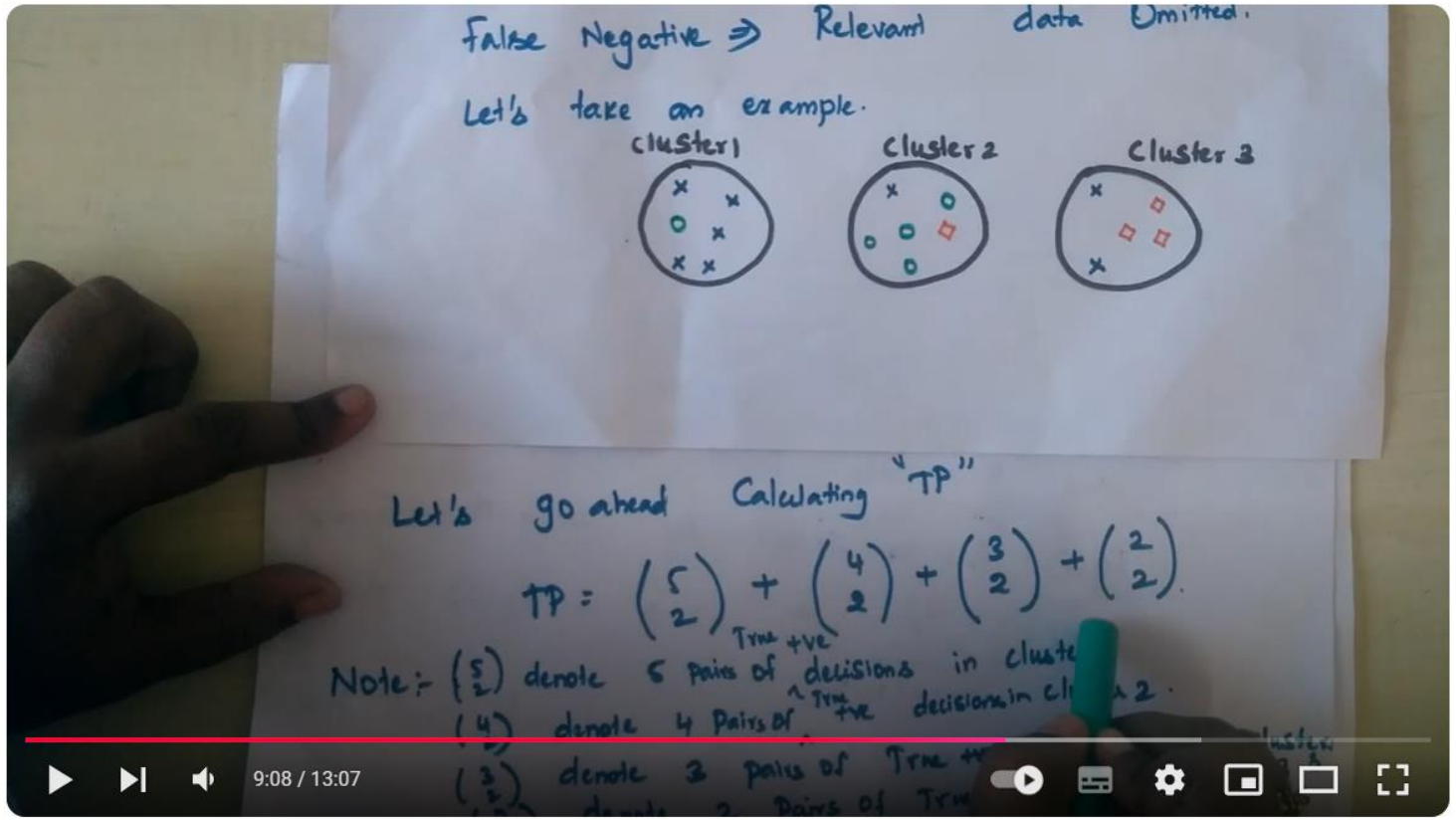


TP is the number of pairs that are in the same cluster in both *U* and *V* (true positives)
TN is the number of pairs that are in the different clusters in both *U* and *V* (true negatives)
FN is the number of pairs that are in the different clusters in *U* but the same cluster in *V* (false negatives)
FP is the number of pairs that are in the same cluster in *U* but different clusters in *V* (false positives)

$$R = \frac{TP + TN}{TP + FP + FN + TN} \Rightarrow ARI = \frac{\text{index} - \text{expected index}}{\text{max index} - \text{expected index}}$$



full details and explanations for this example ☺



Lecture 17- Rand index Calculation

Subalalitha C N
5,29 K de abonați
Abonează-te
226 Trimite Descarcă

2) **NMI** (normalized mutual information)

1 / 15 53%

Normalized Mutual Information

Estimating Clustering Quality

Normalized Mutual Information

- Normalized Mutual Information:

$$NMI(Y,C) = \frac{2 \times I(Y;C)}{[H(Y) + H(C)]}$$

where,

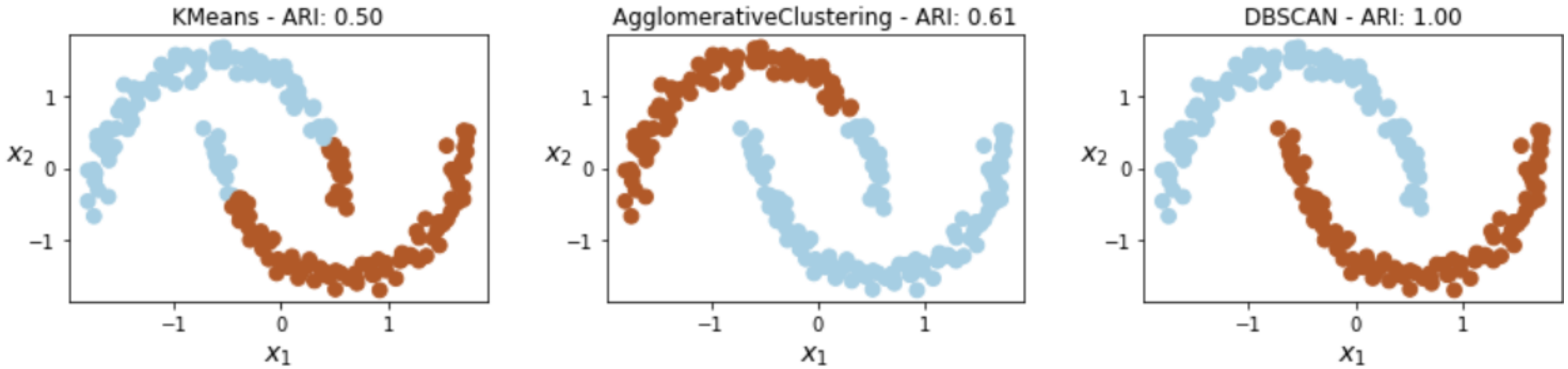
1) Y = class labels
2) C = cluster labels
3) H(.) = Entropy
4) I(Y;C) = Mutual Information b/w Y and C

Note: All logs are base-2.

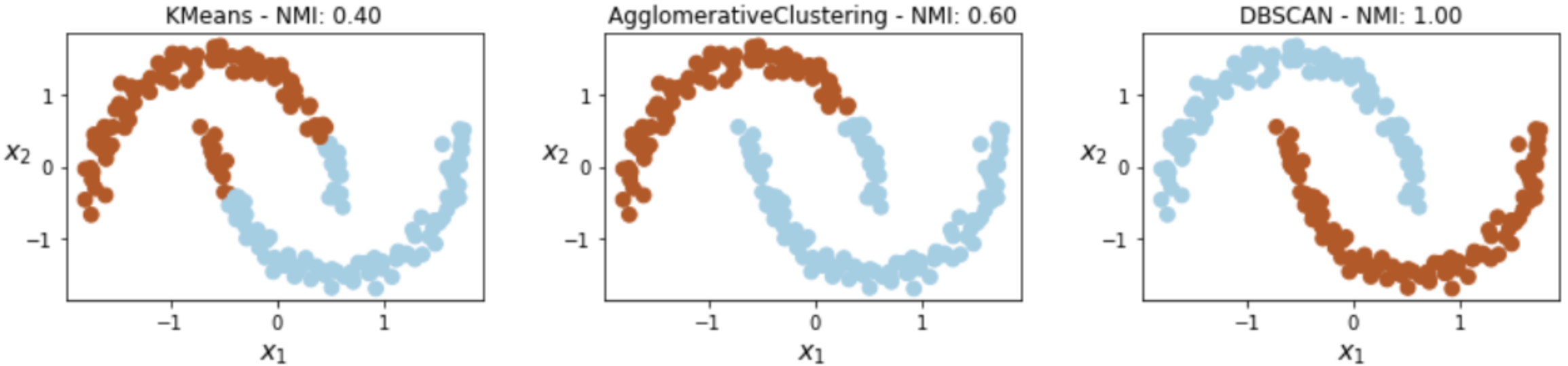
https://course.ccs.neu.edu/cs6140sp15/7_locality_cluster/Assignment-6/NMI.pdf

Code part

adjusted_rand_score() function in *scikit-learn*



normalized_mutual_info_score() function in *scikit-learn*



Exercises

Laboratory 6 – Exercises

1. Use the following dataset:

```
X, y = make_blobs(
        n_samples=400,
        centers=4,
        cluster_std=1.25,
        random_state=42
       ),
```

in order to apply the original K-means clustering algorithm. Run the algorithm for 40 iterations and plot the model's decision boundaries.

2. The following function is used to create a dataset with 2 features:

```
def Circle(r,n):
   return [(math.cos(2*math.pi/n*x)*r+
           +np.random.normal(-30,30),math.sin(2*math.pi/n*x)*r+
           +np.random.normal(-30,30)) for x in range(1,n+1)]
```

This function takes the radius and number of data points as arguments and returns an array of data points which when plotted forms a circle. In order to apply clustering algorithms, three concentric circles are created, and finally noise is added to the data as follows:

```
df=pd.DataFrame(Circle(500,1000))
df=df.append(Circle(300,700))
df=df.append(Circle(100,300))

X=df.append([(np.random.randint(-600,600),
              np.random.randint(-600,600)) for i in range(300)])
```

Perform the following:

- ➢ plot the dataset;
- ➢ apply the K-means++ algorithm and plot the decision boundaries;
- ➢ apply the agglomerative clustering algorithm with complete linkage and visualize the clusters;
- ➢ apply the DBSCAN algorithm with `eps=35` and `min_samples=6`, and plot the clusters.

Solutions

1. Use the following dataset:

```
X, y = make_blobs(
        n_samples=400,
        centers=4,
        cluster_std=1.25,
        random_state=42
       ),
```

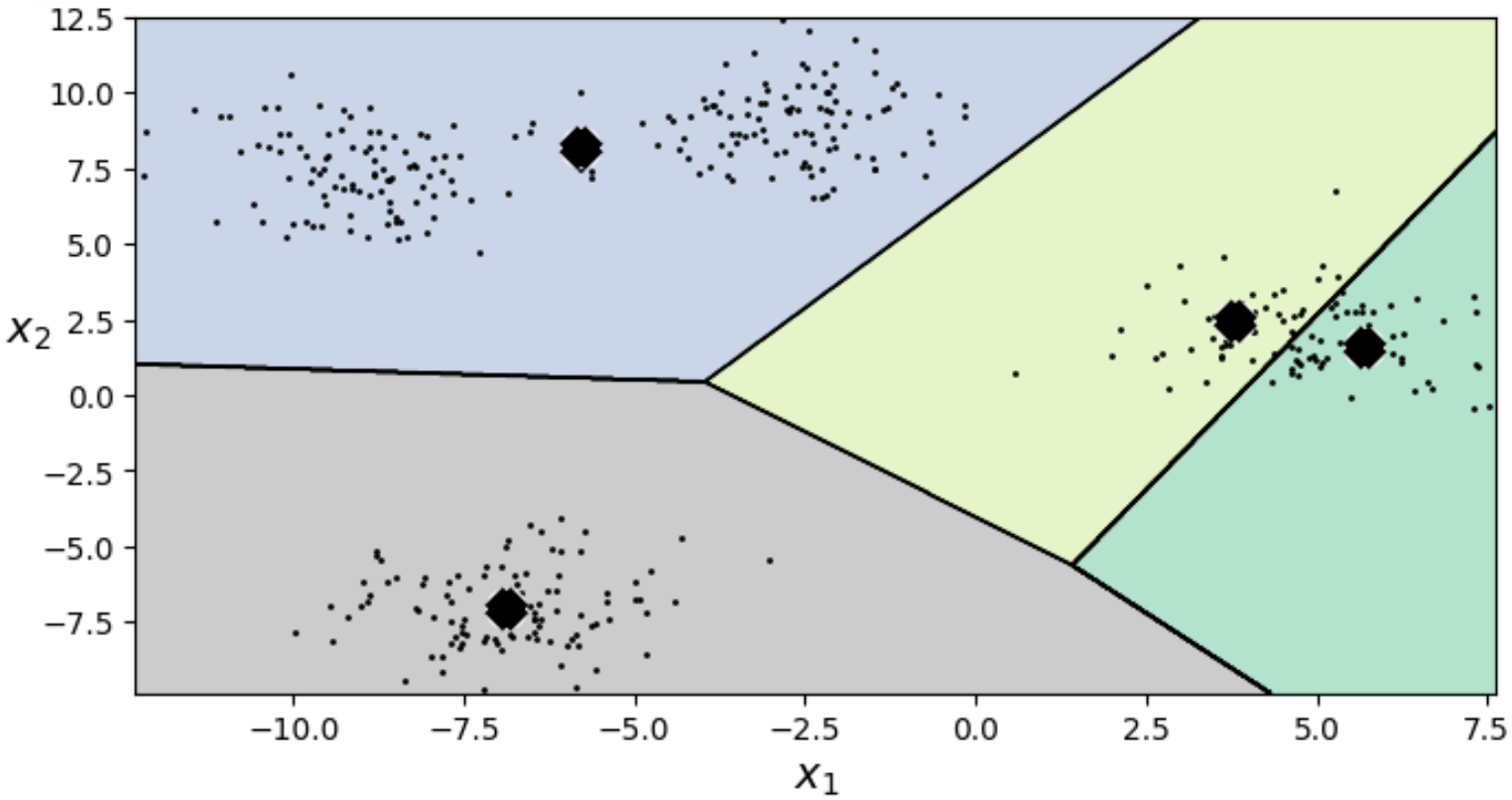
in order to apply the original K-means clustering algorithm. Run the algorithm for 40 iterations and plot the model's decision boundaries.

NOTE: the algorithm="full" parameter was valid in older versions of scikit-learn, but it has since been deprecated. You can resolve this by replacing "full" with "lloyd"

```python
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

# Generate a dataset of 400 samples with 4 centers (clusters) and specified standard deviation
X, y = make_blobs(n_samples=400, centers=4, cluster_std=1.25, random_state=42)

# Initialize KMeans with parameters for the original K-means algorithm
#   - n_clusters=4: we want 4 clusters
#   - init="random": centers are initialized randomly
#   - n_init=1: the algorithm will run only once (no multiple initializations)
#   - algorithm="lloyd": specifying the original K-means algorithm (Lloyd's algorithm)
#   - max_iter=40: maximum number of iterations is set to 40
#   - random_state=42: ensures reproducibility of the results
kmeans = KMeans(n_clusters=4, init="random", n_init=1, algorithm="lloyd", max_iter=40, random_state=42)
kmeans.fit(X) # Fit the KMeans model to the generated data
```

```python
import matplotlib.pyplot as plt
import numpy as np

def plot_data(X): # Function to plot the data points
    plt.plot(X[:, 0], X[:, 1], 'k.', markersize=2) # Plot each data point as a small dot

def plot_centroids(centroids, weights=None, circle_color='w', cross_color='k'): # Function to plot the centroids of the clusters
    if weights is not None:
        centroids = centroids[weights > weights.max() / 10] # Filter centroids with significant weights if provided
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='o', s=35, linewidths=8,
                color=circle_color, zorder=10, alpha=0.9) # Plot circles around centroids
    plt.scatter(centroids[:, 0], centroids[:, 1],
                marker='x', s=35, linewidths=12,
                color=cross_color, zorder=11, alpha=1) # Plot crosses at centroids

def plot_decision_boundaries(clusterer, X, resolution=1000, show_centroids=True): # Function to plot decision boundaries for the KMeans clusters
    mins = X.min(axis=0) - 0.1
    maxs = X.max(axis=0) + 0.1  # Determine the min and max for plotting range with a small margin
    xx, yy = np.meshgrid(np.linspace(mins[0], maxs[0], resolution),
                         np.linspace(mins[1], maxs[1], resolution)) # Create a meshgrid for plotting decision boundaries
    Z = clusterer.predict(np.c_[xx.ravel(), yy.ravel()]) # Predict cluster labels for each point on the grid
    Z = Z.reshape(xx.shape) # Reshape to match the meshgrid shape

    plt.contourf(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                 cmap="Pastel2") # Plot filled contours for decision regions
    plt.contour(Z, extent=(mins[0], maxs[0], mins[1], maxs[1]),
                linewidths=1, colors='k') # Plot contour lines between regions
    plot_data(X) # Plot the original data points
    if show_centroids:
        plot_centroids(clusterer.cluster_centers_) # Plot cluster centroids if required

    plt.xlabel("$x_1$", fontsize=14) # Label the axes for clarity
    plt.ylabel("$x_2$", fontsize=14, rotation=0)

plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X)
plt.show()
```

Result:



2. The following function is used to create a dataset with 2 features:

```
def Circle(r,n):
    return [(math.cos(2*math.pi/n*x)*r+
            +np.random.normal(-30,30),math.sin(2*math.pi/n*x)*r+
            +np.random.normal(-30,30)) for x in range(1,n+1)]
```

This function takes the radius and number of data points as arguments and returns an array of data points which when plotted forms a circle. In order to apply clustering algorithms, three concentric circles are created, and finally noise is added to the data as follows:

```
df=pd.DataFrame(Circle(500,1000))
df=df.append(Circle(300,700))
df=df.append(Circle(100,300))

X=df.append([(np.random.randint(-600,600),
              np.random.randint(-600,600)) for i in range(300)])
```

Perform the following:

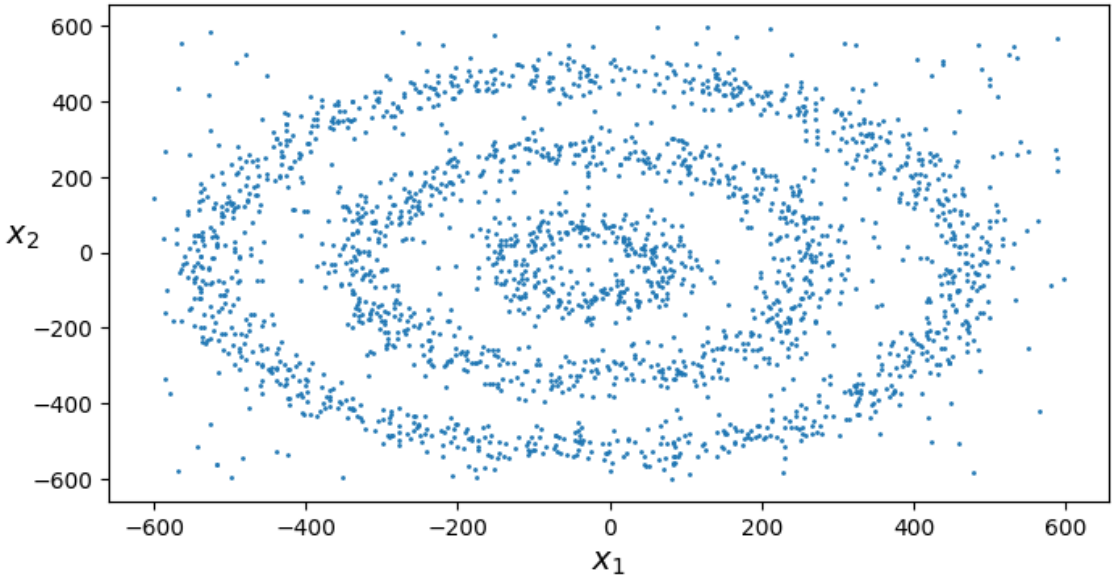
- ➢ plot the dataset;
- ➢ apply the K-means++ algorithm and plot the decision boundaries;
- ➢ apply the agglomerative clustering algorithm with complete linkage and visualize the clusters;
- ➢ apply the DBSCAN algorithm with `eps=35` and `min_samples=6`, and plot the clusters.

NOTE: the **append** method was deprecated in recent versions of **pandas**. We can use **pd.concat** instead, which is now the recommended approach

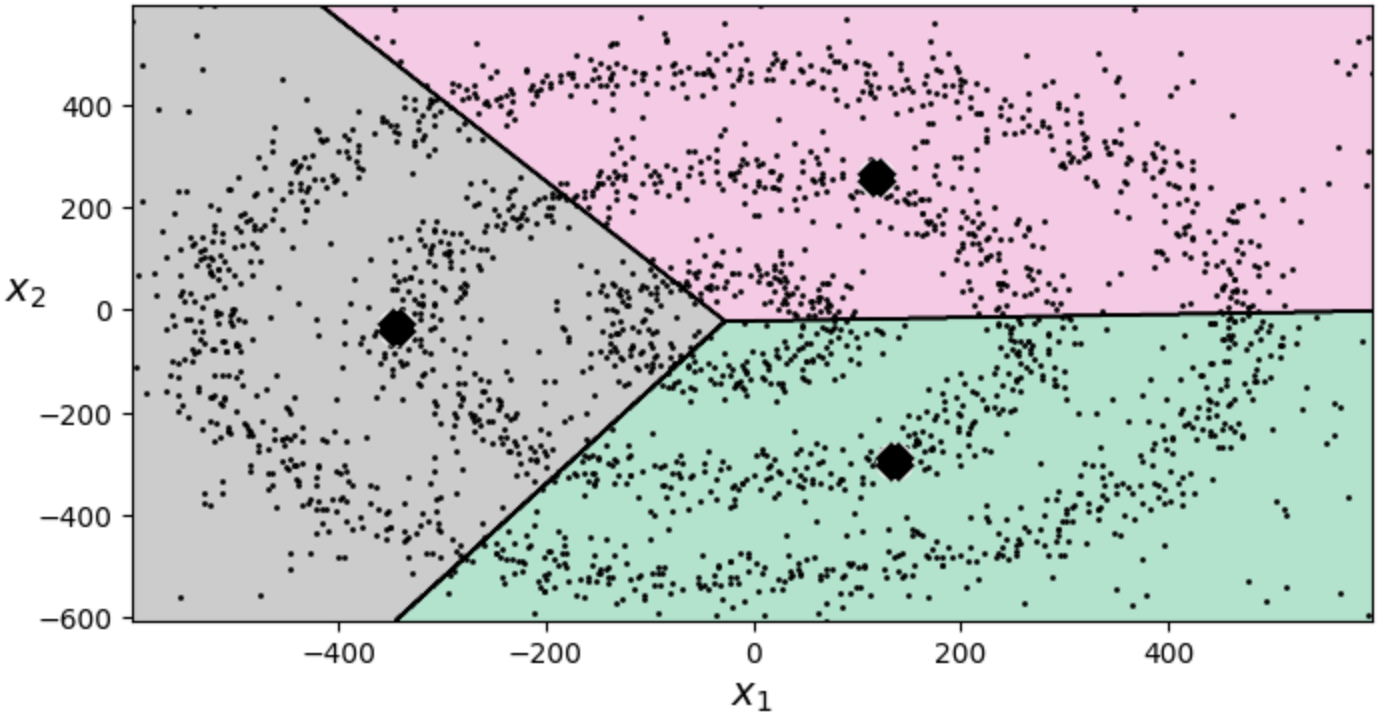
```
import math
import pandas as pd

def Circle(r,n):  # Define the Circle function to generate points in a circular distribution
    return [(math.cos(2*math.pi/n*x)*r
            +np.random.normal(-30,30),math.sin(2*math.pi/n*x)*r
            +np.random.normal(-30,30)) for x in range(1,n+1)]

# Create DataFrames for each circle with different radius and number of points
df1 = pd.DataFrame(Circle(500, 1000))
df2 = pd.DataFrame(Circle(300, 700))
df3 = pd.DataFrame(Circle(100, 300))

# Concatenate the three circles into a single DataFrame
df = pd.concat([df1, df2, df3], ignore_index=True)
```

```python
# Add noise points randomly distributed across a specified range
noise = pd.DataFrame([(np.random.randint(-600, 600), np.random.randint(-600, 600)) for i in range(300)])

# Combine the circular data and noise data
X = pd.concat([df, noise], ignore_index=True)

X=np.array(X) # Convert the DataFrame X to a NumPy array for easier manipulation in clustering and plotting
```

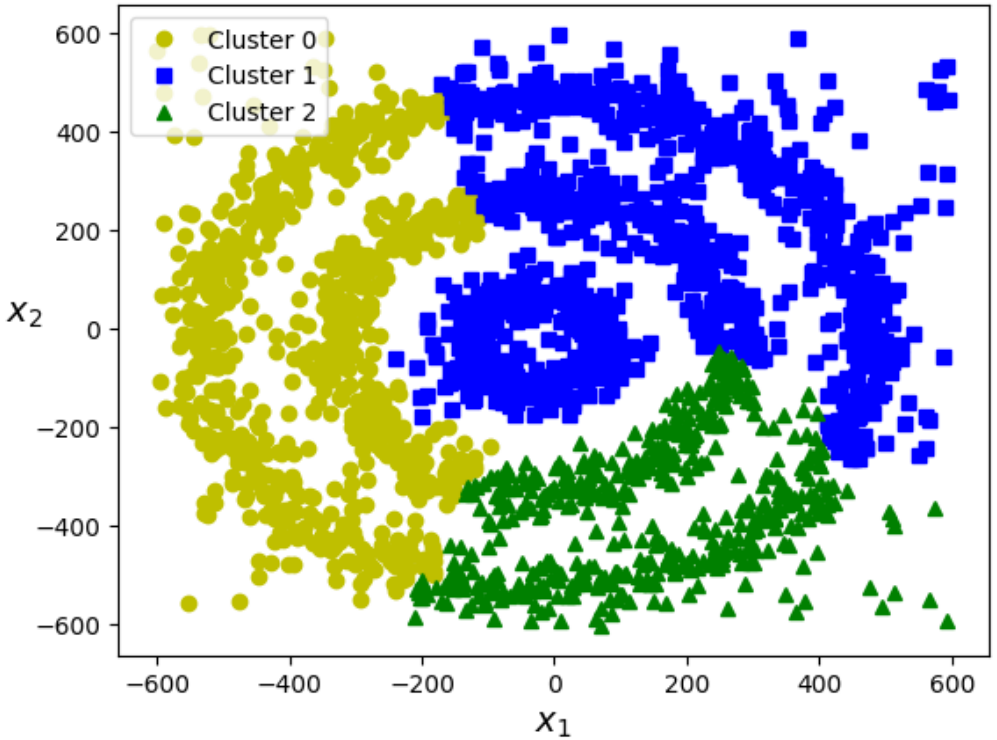
```python
# Plot the dataset
import matplotlib.pyplot as plt

def plot_clusters(X): # Define a function to plot the clusters
    plt.scatter(X[:, 0], X[:, 1], s=1) # Scatter plot of data points with very small marker size (s=1)
    plt.xlabel("$x_1$", fontsize=14) # Label the x-axis for the plot
    plt.ylabel("$x_2$", fontsize=14, rotation=0) # Label the y-axis for the plot, with no rotation for readability

plt.figure(figsize=(8, 4))
plot_clusters(X)
plt.show()
```



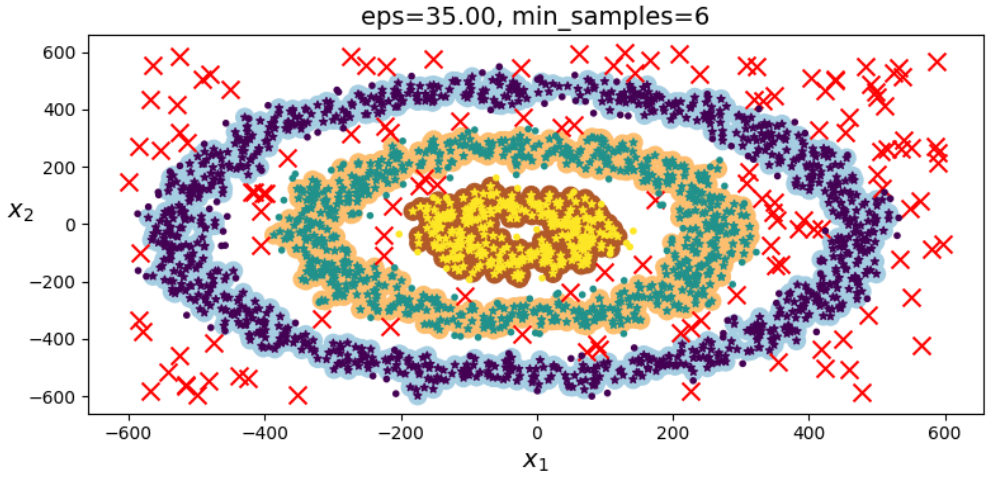
```python
kmeans = KMeans(n_clusters=3, init="k-means++", random_state=42) # Apply K-means++ clustering algorithm with 3 clusters
kmeans.fit(X)

# Plot the decision boundaries for K-means++
plt.figure(figsize=(8, 4))
plot_decision_boundaries(kmeans, X) # Use the plot_decision_boundaries function to visualize boundaries
plt.show()
```



```python
from sklearn.cluster import AgglomerativeClustering # Apply Agglomerative Clustering with complete linkage

agg = AgglomerativeClustering(n_clusters=3, linkage="complete") # Initialize the model
clusters = agg.fit_predict(X) # Fit and predict the clusters for each data point in X

# Plot the clusters formed by Agglomerative Clustering
plt.plot(X[clusters==0, 0], X[clusters==0, 1], "yo") # Plot data points for Cluster 0 in yellow circles
plt.plot(X[clusters==1, 0], X[clusters==1, 1], "bs") # Plot data points for Cluster 1 in blue squares
plt.plot(X[clusters==2, 0], X[clusters==2, 1], "g^") # Plot data points for Cluster 2 in green triangles
plt.legend(["Cluster 0", "Cluster 1", "Cluster 2"], loc="best")
plt.xlabel("$x_1$", fontsize=14)
plt.ylabel("$x_2$", fontsize=14, rotation=0)
plt.show()
```



```python
from sklearn.cluster import DBSCAN # Apply DBSCAN clustering algorithm

dbscan = DBSCAN(eps=35, min_samples=6) # Initialize DBSCAN with specified parameters
dbscan.fit(X) # Fit the DBSCAN model to the data

# Plot the clusters formed by DBSCAN
def plot_dbscan(dbscan, X, size):
  # Create masks for core points, noise points, and non-core points
    core_mask = np.zeros_like(dbscan.labels_, dtype=bool)
    core_mask[dbscan.core_sample_indices_] = True # Mask for core points
    noise_mask = (dbscan.labels_ == -1) # Mask for noise points
    non_core_mask = ~(core_mask | noise_mask) # Mask for non-core (border) points

    # Separate data points into core, noise, and non-core
    cores = dbscan.components_
    noise = X[noise_mask]
    non_cores = X[non_core_mask]

    plt.scatter(cores[:, 0], cores[:, 1], c=dbscan.labels_[core_mask], marker='o', s=size, cmap="Paired") # Plot core points, colored by their cluster label
    plt.scatter(cores[:, 0], cores[:, 1], marker='*', s=20, c=dbscan.labels_[core_mask]) # Highlight core points with a star marker
    plt.scatter(noise[:, 0], noise[:, 1], c="r", marker="x", s=100) # Plot noise points in red with an "x" marker
    plt.scatter(non_cores[:, 0], non_cores[:, 1], c=dbscan.labels_[non_core_mask], marker=".") # Plot non-core points with a small dot
    plt.xlabel("$x_1$", fontsize=14)
    plt.ylabel("$x_2$", fontsize=14, rotation=0)
    plt.title("eps={:.2f}, min_samples={}".format(dbscan.eps, dbscan.min_samples), fontsize=14) # Add a title with DBSCAN parameters for reference

plt.figure(figsize=(8, 4))
plot_dbscan(dbscan, X, size=100)

plt.tight_layout()
plt.show()
```



Thank you !